Modeling and Implementation of a Photovoltaic cell battery charger system

by

Mehmet Gök

B.S., Electrical and Electronics Engineering, Boğaziçi University, 2020

M.S., Electrical and Electronics Engineering, Boğaziçi University, 2026

Submitted to the Institute for Graduate Studies in
Science and Engineering in partial fulfillment of
the requirements for the degree of
Master of Science

Graduate Program in Electrical and Electronics Engineering
Boğaziçi University
2026

Modeling and Implementation of a Photovoltaic cell battery charger system

APPROVED BY:

Prof. Günhan Dündar (Thesis Supervisor)	.....
Assoc. Prof. Name Surname	.....
Assist. Prof. Name Surname	.....
Name Surname, Ph.D.	.....

DATE OF APPROVAL: DD.MM.YYYY

ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to my supervisor, Prof. Günhan Dündar, for his invaluable guidance and support throughout this research. I am also thankful to my family and friends for their unwavering encouragement and patience during this journey.

ABSTRACT

Modeling and Implementation of a Photovoltaic cell battery charger system

This thesis presents the modeling and implementation of a photovoltaic cell battery charger system. The study focuses on the design of a fuzzy logic controller to optimize the charging process while ensuring maximum power point tracking (MPPT). Simulation and experimental results demonstrate the effectiveness of the proposed system in achieving high efficiency and reliability under varying conditions.

ÖZET

Fotovoltaik Hücre Batarya Şarj Sistemi Modelleme ve Uygulaması

Bu tez, bir fotovoltaik hücre batarya şarj sisteminin modellenmesi ve uygulanmasını sunmaktadır. Çalışma, şarj işlemini optimize etmek ve maksimum güç noktası takibini (MPPT) sağlamak için bir bulanık mantık denetleyicisinin tasarımına odaklanmaktadır. Simülasyon ve deneysel sonuçlar, önerilen sistemin değişen koşullar altında yüksek verimlilik ve güvenilirlik sağlama konusundaki etkinliğini göstermektedir.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS . . . iii
ABSTRACT . . . iv
ÖZET . . . v
LIST OF FIGURES . . . viii
LIST OF TABLES . . . x
LIST OF SYMBOLS . . . xi
LIST OF ACRONYMS/ABBREVIATIONS . . . xii
1. INTRODUCTION . . . 1
1.1. Renewable Energy and Photovoltaic Systems . . . 1
1.2. Maximum Power Point Tracking Algorithms . . . 1
1.2.1. Perturb and Observe (P&O) . . . 2
1.2.2. Incremental Conductance (IncCond) . . . 2
1.2.3. Fuzzy Logic Control (FLC) . . . 2
1.3. Proposed Solution . . . 3
2. MATLAB MODEL . . . 4
2.1. MATLAB Model Overview . . . 4
2.2. Photovoltaic Cell Model . . . 5
2.3. Buck Converter . . . 6
2.4. Fuzzy Logic Controller . . . 7
2.4.1. Sampling . . . 8
2.4.2. Rule Firing . . . 9
2.4.3. Duty Adjustment . . . 10
2.5. Simulation Results . . . 11
3. CADENCE MODEL IMPLEMENTATION . . . 14
3.1. PV Model Implementation in Verilog-A . . . 15
3.2. Fuzzy Logic Controller . . . 19
3.2.1. Fuzzification . . . 19
3.2.2. Inference . . . 21

3.2.3. Defuzzification 24
3.2.4. Duty Integration and Bumpless Transfer 26
3.3. Simulation Results 30
3.4. System Block Diagram 31
4. ANALOG IMPLEMENTATION 33
4.1. ADC 33
4.1.1. Sample and Hold Switch 34
4.1.2. CapDAC 35
4.1.3. Preamplifier 40
4.1.3.1. Simulation Results 41
4.1.4. Comparator 44
4.1.4.1. Offset and Noise 45
4.1.5. SAR Logic 46
4.1.5.1. Window Generator 47
4.1.5.2. Comparator Clock Generator 48
4.1.5.3. Bit Generator 49
4.1.6. ADC Top-Level Performance 49
4.2. DAC 51
4.3. LDO 51
4.4. Duty Cycle Generator 52
5. DIGITAL IMPLEMENTATION 53
5.1. Fuzzy Logic 53
5.2. SPI Interface 53
6. CONCLUSION 54
REFERENCES 55
APPENDIX A: APPENDIX A 57

LIST OF FIGURES

1.1 Fuzzy Logic PV Charger Overview. 3
2.1 MATLAB Model Block Diagram. 4
2.2 PV Cell I-V and P-V Characteristics. 6
2.3 Fuzzy Logic Controller Block Diagram. 8
2.4 Overall MATLAB simulation results showing the three performance phases. 12
2.5 Zoomed-in views of transient events. 13
3.1 System Layout in Cadence. 14
3.2 I-V and P-V Characteristics of the Verilog-A PV Model. 18
3.3 Cadence Mixed-Signal Simulation Results showing MPP Tracking. 30
3.4 System Block Diagram. 31
4.1 Fully Differential 12-bit Asynchronous SAR ADC Architecture. . . 33
4.2 Bootstrapped Sample and Hold Switch Implementation. 34
4.3 DFT of the Signal Passed Through the Sample and Hold Switch. . 35
4.4 CapDAC Linearity Simulation Results. 39
4.5 Fully Differential Preamplifier with CMFB Circuit. 41
4.6 CMFB Loop Stability Simulation. 42
4.7 Output Common Mode Voltage Monte Carlo Simulation. 42
4.8 Differential Open Loop AC Response. 43
4.9 Input Referred Offset Monte Carlo Simulation. 43
4.10 Input Referred Noise Simulation. 44
4.11 Comparator Input Referred Offset Monte Carlo Simulation. 45
4.12 Comparator Input Referred Noise Simulation. 46
4.13 Asynchronous SAR Logic Overview with Level Shifters. 47
4.14 Window Generator Logic Implementation. 48
4.15 Asynchronous Comparator Clock Generation Logic. 48
4.16 Timing Diagram of the Asynchronous Operation. 49
4.17 Bit Generator Logic Implementation. 49

4.18 4096-Point FFT of the ADC Output. 51

LIST OF TABLES

2.1 Buck Converter Specifications 7

2.2 Fuzzy Logic Safety Rules 11

2.3 Fuzzy Logic Rules for Optimal Range 11

3.1 Comparison of PV Model Parameters 18

4.1 Sample and Hold Switch Performance Metrics (Input: 0-5 V FS) . 35

4.2 Preamplifier Performance Summary 44

4.3 Comparator Performance Summary 46

4.4 ADC Performance Across Process Corners 51

LIST OF SYMBOLS

V_{pv}	Photovoltaic panel output voltage
I_{pv}	Photovoltaic panel output current
P_{pv}	Photovoltaic panel output power
V_b	Battery voltage
η	Efficiency
τ	Time constant
Δ	Change or differential

LIST OF ACRONYMS/ABBREVIATIONS

ADC	Analog-to-Digital Converter
CC	Constant Current
DAC	Digital-to-Analog Converter
FLC	Fuzzy Logic Controller
I-V	Current-Voltage
LDO	Low Drop Out Regulator
MPPT	Maximum Power Point Tracking
P-V	Power-Voltage
PV	Photovoltaic
PWM	Pulse Width Modulation
S/H	Sample and Hold
SoC	State of Charge
SPI	Serial Peripheral Interface

1. INTRODUCTION

1.1. Renewable Energy and Photovoltaic Systems

The global demand for energy is rising rapidly, driven by population growth and industrialization. Concurrently, environmental concerns regarding fossil fuels have accelerated the transition towards renewable energy sources. Among these, solar energy stands out as a clean, abundant, and sustainable resource. Photovoltaic (PV) systems, which convert sunlight directly into electricity using semiconductor materials, have become a dominant technology in the renewable energy sector.

However, the efficiency of energy conversion in PV systems is heavily dependent on environmental conditions. The power output of a PV panel is non-linear and varies significantly with solar irradiance and ambient temperature. Unlike conventional power sources, a PV array does not have a constant internal impedance. Consequently, to extract the maximum available power, the load impedance must be dynamically matched to the source impedance. This necessity leads to the implementation of Maximum Power Point Tracking (MPPT) controllers.

1.2. Maximum Power Point Tracking Algorithms

Maximum Power Point Tracking (MPPT) is a critical technique in photovoltaic (PV) systems used to maximize power extraction under all conditions. The P-V characteristic of a solar panel is non-linear and varies with solar irradiance and temperature. There is a unique point on the P-V curve, known as the Maximum Power Point (MPP), where the PV system produces maximum power. MPPT algorithms adjust the electrical operating point of the modules to keep them at the MPP. Several algorithms have been developed to achieve this, varying in complexity, sensor requirements, convergence speed, and cost.

1.2.1. Perturb and Observe (P&O)

The Perturb and Observe (P&O) algorithm is one of the most widely used methods due to its simplicity and ease of implementation. It operates by periodically perturbing (increasing or decreasing) the terminal voltage of the PV array and comparing the PV output power with that of the previous cycle. If the power increases, the perturbation is continued in the same direction. If the power decreases, the direction is reversed. While simple, P&O suffers from oscillations around the MPP in steady state and can track in the wrong direction during rapidly changing atmospheric conditions.

1.2.2. Incremental Conductance (IncCond)

The Incremental Conductance (IncCond) method is based on the fact that the slope of the PV array power curve is zero at the MPP, positive on the left of the MPP, and negative on the right. It compares the instantaneous conductance (I/V) with the incremental conductance (dI/dV). Although IncCond offers better tracking accuracy and stability compared to P&O, particularly under rapidly changing conditions, it requires more complex control circuitry and higher computational power.

1.2.3. Fuzzy Logic Control (FLC)

Fuzzy Logic Control (FLC) has emerged as a powerful alternative for MPPT, especially in systems where the mathematical model is difficult to derive or is highly non-linear. FLC operates based on linguistic rules and does not require precise knowledge of the system parameters. It is robust against parameter variations and environmental noise. Compared to P&O and IncCond, FLC offers faster convergence and reduced oscillations around the MPP, making it highly suitable for battery charging applications where efficiency and battery health are paramount.

1.3. Proposed Solution

Based on the comparison of MPPT techniques presented in the previous section, this thesis proposes a photovoltaic battery charger system that employs a Fuzzy Logic Controller (FLC). While P&O is simple, it suffers from oscillations, and IncCond, while accurate, is computationally intensive. FLC offers a balance of robustness, fast convergence, and adaptability to the non-linear characteristics of PV panels without requiring complex mathematical modeling.

The proposed system integrates the PV panel with a DC-DC buck converter controlled by the FLC algorithm to charge a battery efficiently. This approach ensures optimal energy harvesting even under rapidly changing atmospheric conditions.

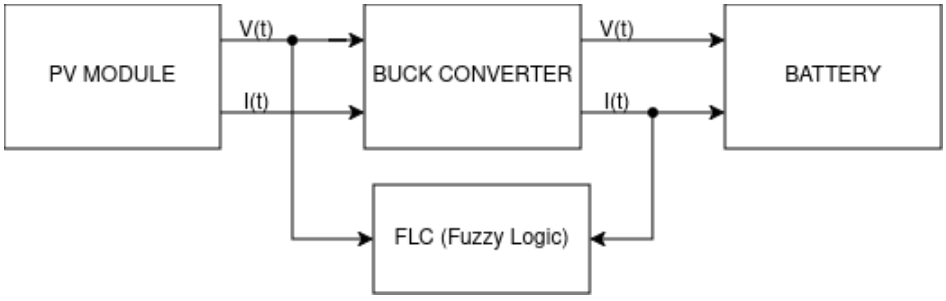


Figure 1.1. Fuzzy Logic PV Charger Overview.

2. MATLAB MODEL

2.1. MATLAB Model Overview

The design and verification of the proposed photovoltaic battery charger system were initially conducted using the MATLAB/Simulink environment. Simulink provides a comprehensive platform for Model-Based Design, allowing for the simulation of complex dynamic systems before physical implementation.

One of the primary advantages of using Simulink for this application is the availability of the Simscape Electrical library. This library offers pre-built, high-fidelity models for power electronic components, such as MOSFETs, diodes, inductors, and capacitors, as well as specialized sources like the Photovoltaic Array block. The built-in PV array model allows for precise definition of the panel's characteristics and enables the simulation of varying environmental conditions, such as changes in solar irradiance and temperature, which are crucial for testing MPPT algorithms.

Furthermore, MATLAB's Fuzzy Logic Toolbox facilitates the design and visualization of the Fuzzy Inference System (FIS). This toolbox allows for the intuitive definition of membership functions and rules, which can then be directly integrated into the Simulink simulation via the Fuzzy Logic Controller block. This integration enables rapid prototyping and tuning of the control parameters to achieve optimal tracking performance and stability.

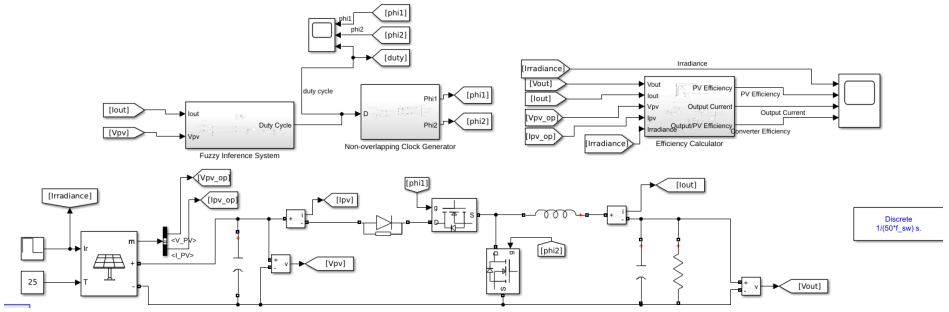


Figure 2.1. MATLAB Model Block Diagram.

2.2. Photovoltaic Cell Model

A fundamental component of the proposed system is the photovoltaic (PV) cell, which converts solar energy into electrical energy. To accurately simulate and control the charging process, a mathematical model representing the PV cell's behavior is essential. This section details the model used, specifically focusing on its non-linear current-voltage (I-V) and power-voltage (P-V) characteristics.

The output power of a PV panel is not constant; it varies significantly with environmental conditions, primarily solar irradiance and temperature. As shown in the P-V characteristics, there exists a single operating point for any given set of conditions where the power extraction is maximized. This point is referred to as the Maximum Power Point (MPP). Because the location of the MPP shifts as irradiance or temperature changes, a static connection between the panel and the load is inefficient. Therefore, it is imperative to employ Maximum Power Point Tracking (MPPT) techniques. MPPT ensures that the load impedance seen by the source is adjusted to match the optimal operating point, thereby maximizing the efficiency of energy harvesting.

Figure 2.2 illustrates the I-V and P-V characteristics of the modeled PV cell. The plot explicitly displays curves for three different irradiance levels, highlighting how the short-circuit current and the maximum power point scale with the intensity of the incident sunlight.

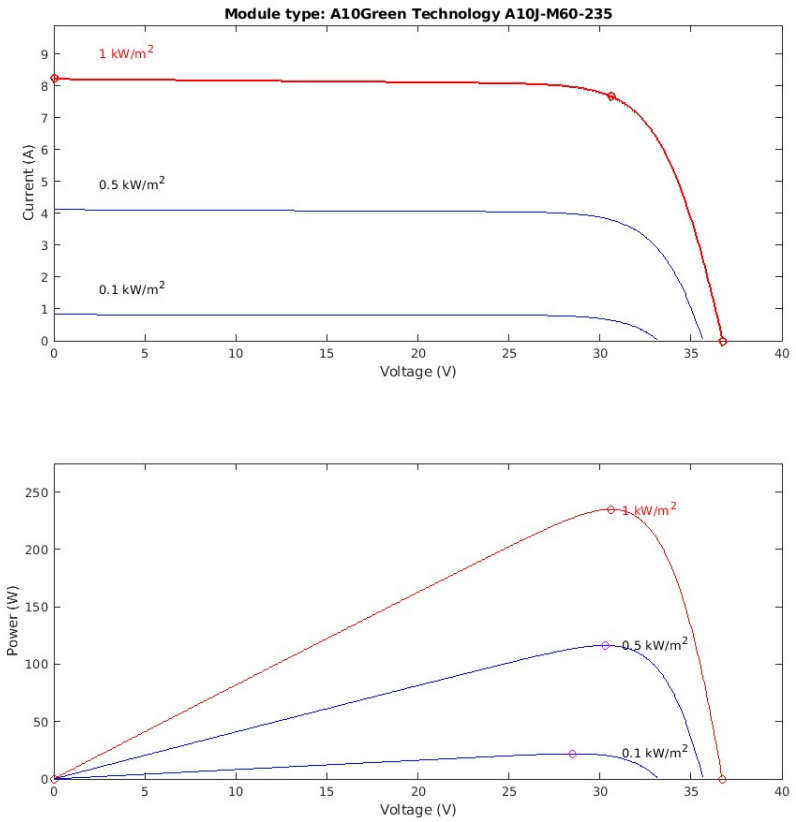


Figure 2.2. PV Cell I-V and P-V Characteristics.

2.3. Buck Converter

The buck converter is a fundamental DC-DC topology used to step down a higher input voltage to a lower output voltage while maintaining high efficiency. It operates by chopping the input voltage using a controlled switch (typically a MOSFET) and filtering the resulting signal with an inductor and a capacitor. The ratio of the output voltage to the input voltage is primarily determined by the duty cycle of the switch.

In this specific application, the photovoltaic panel operates at a Maximum Power Point (MPP) voltage of approximately 30 V. However, the system is designed to charge a 12 V battery. Consequently, a buck converter is employed to bridge this voltage difference. This configuration not only steps down the voltage but also allows the

MPPT algorithm to adjust the effective load impedance seen by the PV panel by varying the duty cycle, thereby maximizing power extraction.

The design parameters for the buck converter, as defined in the system's MATLAB model, are presented in Table 2.1. These specifications are derived based on the design guidelines and component selection criteria described in [1–3].

Table 2.1. Buck Converter Specifications

Parameter	**Symbol**	**Value**
Input Voltage (MPP)	V_{in}	30.6 V
Output Voltage (Battery)	V_{out}	14.4 V
Switching Frequency	f_{sw}	500 kHz
Inductance	L	2.6 μH
Input Capacitance	C_{in}	47 μF

2.4. Fuzzy Logic Controller

To address the non-linear characteristics of the PV array and the need for robust tracking under changing environmental conditions, this thesis employs a Fuzzy Logic Controller (FLC). Unlike traditional linear controllers, the FLC does not require an exact mathematical model of the plant, making it highly effective for the time-varying nature of solar irradiance and temperature.

The control algorithm is based on a Sugeno-type fuzzy inference system, which is chosen for its computational efficiency and its suitability for generating a crisp control output, making it well-suited for digital implementation in control systems. The overall process of the FLC can be divided into three main stages: sampling, rule firing, and duty adjustment.

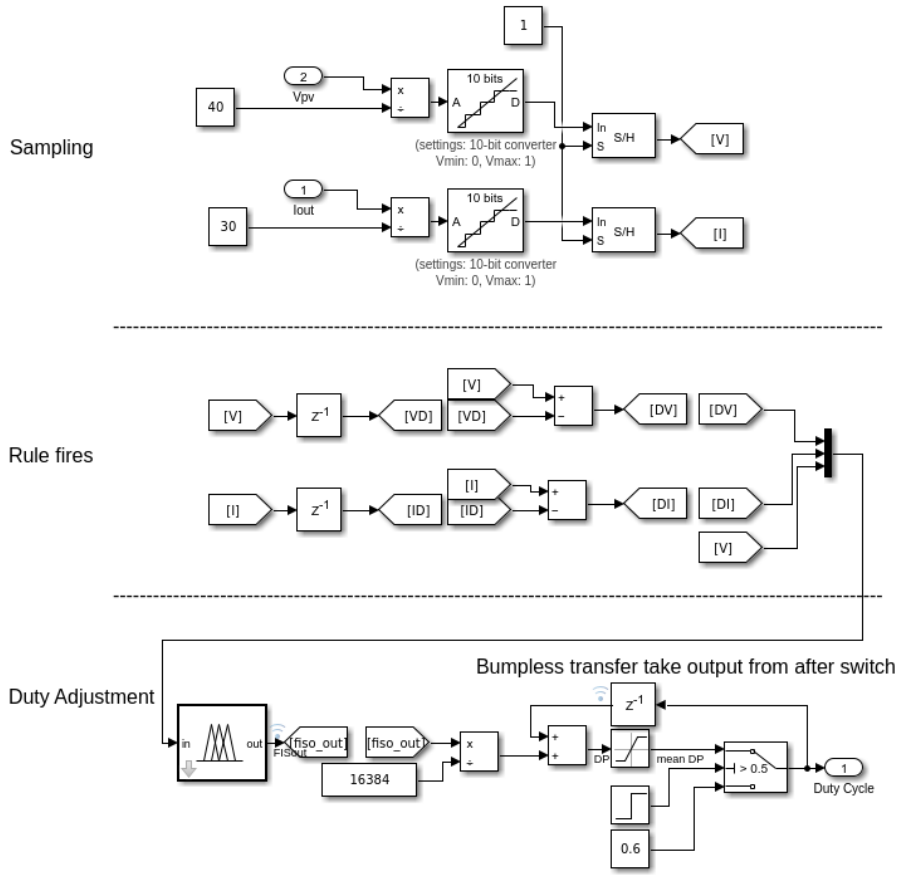


Figure 2.3. Fuzzy Logic Controller Block Diagram.

2.4.1. Sampling

The controller is implemented digitally with a sampling frequency of 20 kHz, as defined in the system parameters. At each sampling instant, the controller reads the PV voltage and current. The ADCs used for this purpose have a 10-bit resolution, which provides $2^{10} = 1024$ discrete levels (counts) to represent the analog signals. This allows for a fine-grained measurement of the PV voltage and current, which is essential for the accuracy of the MPPT algorithm. This sampling frequency is chosen to be sufficiently lower than the 500 kHz switching frequency to avoid aliasing while maintaining a fast transient response.

2.4.2. Rule Firing

The core of the FLC is the rule firing stage, where the necessary change in the duty cycle (ΔD) is determined based on the sampled inputs. This stage processes three key inputs:

(i) **Change in PV Voltage (dV):** The difference in the PV voltage reading between consecutive samples.

(ii) **Change in PV Current (dI):** The difference in the PV current reading between consecutive samples.

(iii) **PV Voltage (V_{pv}):** The instantaneous PV voltage, used to determine the operating region of the PV panel.

The process involves three steps:

- **Fuzzification:** The crisp input values are converted into linguistic variables (e.g., Negative, Zero, Positive for dV and dI; `Low_Voltage`, `Optimal_Range`, `High_Voltage` for V_{pv}) using linear and triangular membership functions.
- **Rule Evaluation:** A set of linguistic rules is evaluated to determine the control action. The rule base, summarized in Tables 2.2 and 2.3, is divided into two main groups: safety rules that make large adjustments when the voltage is too low or too high, and fine-tuning rules for operation near the MPP. A critical rule is that if both dV and dI are zero, the duty cycle is held constant to prevent oscillations at the power peak.
- **Defuzzification:** The output of the Sugeno system is a set of constant values for the change in duty cycle (ΔD), such as `Big_Decrease` (-15), `Decrease` (-1), `Hold` (0), `Increase` (+1), and `Big_Increase` (+15). The final output is a weighted average of the active rule outputs.

These rules provide a two-tiered control strategy. The safety rules (Table 2.2) trigger when the PV voltage moves into the `Low_Voltage` or `High_Voltage` regions.

These regions are defined by membership functions in the fuzzy inference system, as specified in the `matlab_code/create_mppt_fis.m` script. With a 10-bit ADC and a 40V reference, the conversion from a voltage V to an ADC count is given by: $ADC_{count} = (V/40V) \times 1024$.

The MPP for the modeled panel is at 30.6V (approx. ADC count 783). The safety rules are configured for large deviations from this point. For instance, the `Low_Voltage` membership function (see `create_mppt_fis.m`) becomes fully active for ADC counts below 730. This corresponds to a voltage of approximately 28.5V, calculated as $(730/1024) \times 40V$. If the panel voltage drops into this region, the controller applies a `Big_Decrease` to the duty cycle, raising the panel voltage back towards the MPP. Conversely, the `High_Voltage` membership function begins to activate at an ADC count of 840, which corresponds to 32.8V ($(840/1024) \times 40V$). If the voltage exceeds this, the controller applies a `Big_Increase` to lower the panel voltage.

Once the voltage is within the `Optimal_Range`, the fine-tuning rules take over, making small adjustments to zero in on the precise MPP and hold it there, as detailed in Table 2.3.

2.4.3. Duty Adjustment

The final stage is the adjustment of the duty cycle. To ensure a smooth transition between different control states and prevent abrupt changes in the duty cycle, a bumpless transfer mechanism is implemented. This is achieved by taking the duty cycle output from the general duty cycle register ($D[n-1]$) and adding the calculated change (ΔD). This feedback loop ensures that the control signal evolves continuously from the previous state, preventing discontinuities when switching between loop dynamics.

Additionally, the integrator and the scaling of the output provide a damping effect. This lowers the impact of abrupt changes, reducing oscillations and making the system robust against noise and environmental disturbances. In the MATLAB model,

the calculated ΔD is divided by a gain factor (e.g., 16384). This "lowering effect" reduces the magnitude of the duty cycle adjustments, allowing for very fine control granularity in the continuous simulation environment.

Table 2.2. Fuzzy Logic Safety Rules

PV Voltage (V_{pv})	**Output (ΔD)**
`Low_Voltage`	`Big_Decrease`
`High_Voltage`	`Big_Increase`

Table 2.3. Fuzzy Logic Rules for Optimal Range

dI \ dV	**Negative**	**Zero**	**Positive**
Negative	`Decrease`	`Hold`	`Increase`
Zero	Not Defined	`Hold`	Not Defined
Positive	`Increase`	`Hold`	`Decrease`

2.5. Simulation Results

The performance of the proposed fuzzy logic-based MPPT controller was validated through a comprehensive simulation conducted in MATLAB/Simulink, with the overall results depicted in Figure 2.4. The total simulation time was 0.6 seconds, designed to demonstrate the controller's performance during both steady-state and transient conditions. The converter efficiency, calculated as P_{out}/P_{in}, was maintained at approximately 95% throughout the simulation. The PV efficiency, defined as P_{in}/P_{vmax}, is the primary metric for evaluating the MPPT algorithm's effectiveness.

The simulation is divided into three distinct phases, with the critical transient moments highlighted in detail in Figure 2.5.

(i) **Initial Stabilization (0s - 0.2s):** For the first 0.2 seconds, the system operates with a fixed duty cycle. The fuzzy logic controller is inactive, allowing the

converter to stabilize without active MPPT control, as seen in the first section of Figure 2.4.

(ii) **Fuzzy Controller Activation (0.2s):** At 0.2 seconds, the fuzzy logic MPPT controller is enabled. Figure 2.5(a) shows a zoomed-in view of this event. The controller rapidly adjusts the duty cycle to find the maximum power point, achieving over 99% PV efficiency in just 13.4 milliseconds, which highlights its fast convergence speed.

(iii) **Transient Response (0.4s):** At 0.4 seconds, a step change simulates a sudden decrease in solar irradiance. Figure 2.5(b) details this transient. Despite the disturbance causing a momentary dip in PV efficiency to 90%, the fuzzy controller responds swiftly, restoring the efficiency to over 99% in approximately 1 millisecond.

These results confirm the robustness and high performance of the fuzzy logic controller, showcasing its ability to quickly track the MPP and maintain high efficiency, even under rapidly changing conditions.

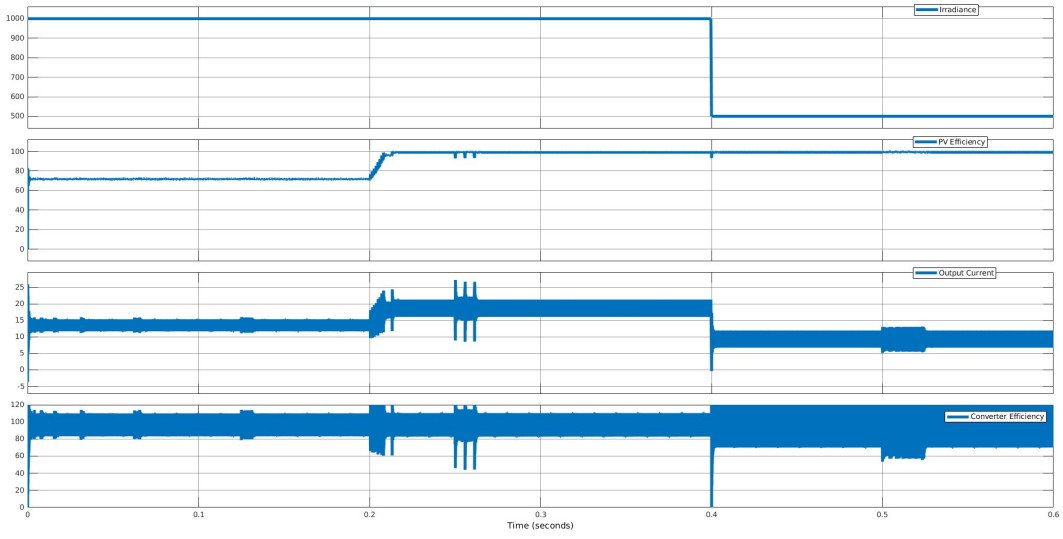
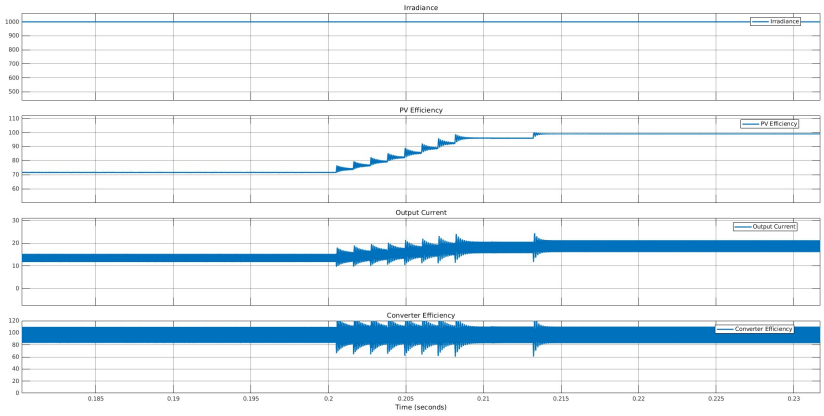
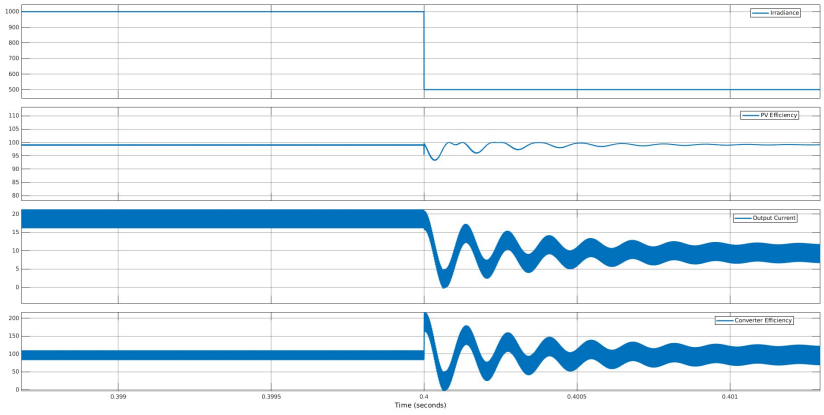


Figure 2.4. Overall MATLAB simulation results showing the three performance phases.



(a) Zoomed-in view of FLC activation at 0.2s.



(b) Zoomed-in view of transient response at 0.4s.

Figure 2.5. Zoomed-in views of transient events.

3. CADENCE MODEL IMPLEMENTATION

The initial design and verification of the fuzzy logic controller were conducted in the MATLAB/Simulink environment, as detailed in Chapter 2. While MATLAB provides a powerful platform for algorithmic development and system-level simulation, the ultimate goal is to create a physical integrated circuit. To move towards a silicon-compatible implementation, the validated model must be migrated from the abstract, high-level MATLAB environment to a hardware description language (HDL) and simulated in an environment that supports circuit-level design and verification, such as Cadence. This chapter details the process of this migration, covering the implementation of the core components in SystemVerilog and Verilog-A, and the verification of the complete system within the Cadence Virtuoso platform.

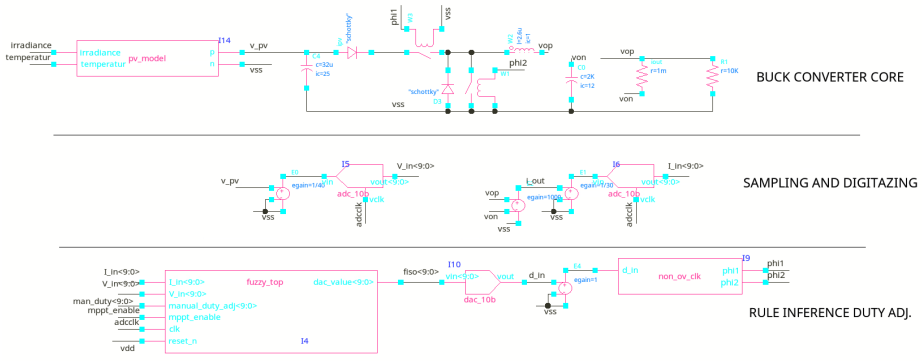


Figure 3.1. System Layout in Cadence.

The overall system layout for the Cadence implementation is shown in Figure 3.1. While some components of the system, such as the ADCs, DACs, and the power switches of the buck converter, are based on standard and well-understood designs, their detailed implementation is not the focus of this chapter. Instead, the following subsections will focus on the more complex and non-straightforward parts of the design: the implementation of the fuzzy inference engine in SystemVerilog and the behavioral modeling of the PV panel in Verilog-A.

3.1. PV Model Implementation in Verilog-A

To create a realistic simulation environment in Cadence, the photovoltaic panel was modeled using Verilog-A. This behavioral model replicates the non-linear I-V and P-V characteristics of the PV cell, allowing for accurate simulation of its interaction with the digitally-controlled buck converter. The model is based on the same mathematical equations used in the MATLAB simulation.

A widely recognized method for modeling PV panels is the five-parameter model, which uses a simplified equivalent circuit to predict the I-V curve. A significant contribution in this area was made by De Soto et al. [4], who developed and validated a method to determine the five key parameters (light-generated current I_L, diode saturation current I_0, ideality factor n, series resistance R_s, and shunt resistance R_{sh}) from standard manufacturer datasheet values. This approach allows for accurate performance prediction across a wide range of operating conditions (irradiance and temperature), which is essential for testing MPPT algorithms.

The Verilog-A implementation, shown below, is based on this five-parameter model. It takes irradiance and temperature as inputs and dynamically calculates the panel's output current based on its terminal voltage, accurately reproducing the non-linear behavior of the PV cell.

Listing 3.1. Verilog-A PV Model Implementation

```
`include "disciplines.vams"
`include "constants.vams"

// Photovoltaic Panel Model
// Implementation for Master's Thesis: PV to Battery Charger
   System
module pv_model (irradiance, temperatur, p, n);
    inout p, n;
    input irradiance, temperatur;
```

```
electrical p, n, irradiance, temperatur;

// --- Array Configuration ---
parameter real Ns_mod  = 1;      // Series modules
parameter real Np_mod  = 1;      // Parallel strings
parameter real N_cell  = 60;     // Cells per module

// --- Model Parameters (Extracted from Simulink/A10Green
   ) ---
parameter real IL_ref  = 8.2383;     // Light current [A]
parameter real I0_ref  = 2.8335e-10; // Saturation
   current [A]
parameter real n_id    = 0.98962;    // Ideality factor
parameter real Rs_ref  = 0.25811;    // Series resistance
    [Ohm]
parameter real Rsh_ref = 196.9278;   // Shunt resistance
   [Ohm]

// --- Physics & Temp Constants ---
parameter real alpha_Isc = 0.00096999; // Temperature
   coefficient [A/C]
parameter real Tref      = 25.0;       // Reference
   temperature [C]
parameter real Eg        = 1.12;       // Bandgap energy
   [eV] for Si

// Internal Variables
real Tc_degC, Tc_K, Tref_K;
real G_suns, IL, I0, Vt, Rs, Rsh, V_diode;

analog begin
    // 1. Environmental Input Scaling
    G_suns  = V(irradiance);
```

```
        Tc_degC = V(temperatur);
        Tc_K    = Tc_degC + 273.15;
        Tref_K  = Tref + 273.15;

        // 2. Variable Calculation
        Vt = (N_cell * Ns_mod * `P_K * Tc_K / `P_Q) * n_id;

        // Adjust for current irradiance and temperature
        IL = (IL_ref + alpha_Isc * (Tc_degC - Tref)) * G_suns
           ;

        // Calculate saturation current temperature
           dependence
        I0 = I0_ref * pow((Tc_K / Tref_K), 3.0) * limexp((
           `P_Q * Eg / (n_id * `P_K)) * (1/Tref_K - 1/Tc_K));

        // Scale parasitic resistances for array size
        Rs  = Rs_ref * Ns_mod / Np_mod;
        Rsh = Rsh_ref * Ns_mod / Np_mod;

        // 3. Kirchhoff's Current Law Implementation
        // Define diode voltage including the drop across
           series resistance
        V_diode = V(p, n) - I(p, n) * Rs;

        // Final output current: Losses (Diode + Shunt) minus
            Source (Photocurrent)
        I(p, n) <+ (I0 * Np_mod * (limexp(V_diode / Vt) - 1))
            +
                (V_diode / Rsh) - (IL * Np_mod);
    end
endmodule
```

The Verilog-A code provides a behavioral model of the PV cell. Figure 3.2 shows the I-V (current-voltage) and P-V (power-voltage) characteristics produced by this model, highlighting the Maximum Power Point (MPP).

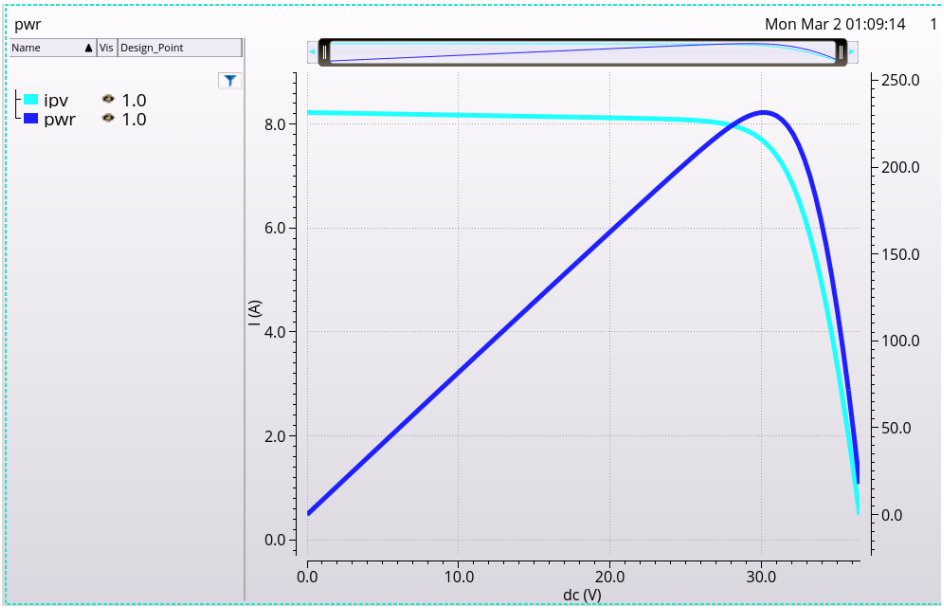


Figure 3.2. I-V and P-V Characteristics of the Verilog-A PV Model.

While the MATLAB/Simulink model serves as the ideal reference, the Verilog-A behavioral model implemented in Cadence exhibits slight deviations in the electrical characteristics. These minor discrepancies arise from the implementation of the five-parameter equations and the analog solver's interpretation. Table 3.1 compares the key parameters of the MATLAB reference model and the Verilog-A implementation.

Table 3.1. Comparison of PV Model Parameters

Parameter	**MATLAB Model**	**Verilog-A Model**
Open Circuit Voltage (V_{oc})	36.72 V	36.5 V
Short Circuit Current (I_{sc})	8.23 A	8.226 A
MPP Voltage (V_{mpp})	30.6 V	30.29 V
Maximum Power (P_{mpp})	235.008 W	231.4 W

3.2. Fuzzy Logic Controller

To prepare the design for hardware implementation, the fuzzy logic controller, previously validated in MATLAB, was implemented as a digital module using SystemVerilog. SystemVerilog is a hardware description and verification language that allows for the creation of synthesizable logic, providing a direct path to a physical integrated circuit. This implementation follows a Sugeno-type fuzzy inference model, which is well-suited for hardware due to its computational efficiency. The process is broken down into three fundamental stages: fuzzification, inference, and defuzzification.

3.2.1. Fuzzification

In the fuzzification stage, the crisp digital inputs from the ADCs, representing the PV panel's voltage (V_{pv}) and current (I_{pv}), are converted into fuzzy linguistic variables. This is achieved by evaluating their degree of membership in predefined sets (e.g., 'Low', 'Optimal', 'High').

The core of this process is the calculation of the membership degree for a given input value. For this controller, triangular membership functions are used, which are defined by three points. A triangular function is composed of two linear segments, meaning the membership degree can be calculated using the simple equation of a line. If the slope and two points of a line are known, any other point is easily calculable.

Given two points (x_1, y_1) and (x_2, y_2), the slope (m) of the line connecting them is:

$$m = \frac{y_2 - y_1}{x_2 - x_1} \tag{3.1}$$

With the slope and a known point, the membership degree (y) for any input (x) can

be calculated using the point-slope formula:

$$y - y_1 = m(x - x_1) \tag{3.2}$$

This linear relationship makes the calculation straightforward to implement in hardware, as shown in the SystemVerilog code below.

Listing 3.2. MF triangle SystemVerilog Implementation

```
module mf_triangle #(
    parameter signed [10:0] A = -383,
    parameter signed [10:0] B = -300,
    parameter signed [10:0] C = -217
)(
    input  logic signed [10:0] x,    // input variable (ADC
       diff)
    output logic [7:0] mu            // membership degree
       (0-255)
);
    // Local parameter for scaling
    localparam logic signed [10:0] SCALE = 11'd255;
    // Internal signals
    logic signed [11:0] num1, num2, den1, den2;
    logic signed [21:0] tmp;
    logic signed [15:0] result;
    always_comb begin
        // Default outputs
        mu = 8'd0;
        // Left slope: A < x < B
        if ((x > A) && (x < B)) begin
            num1 = x - A;
            den1 = B - A;
            tmp = num1 * SCALE;
            // Rounded division: (tmp + den1/2) / den1
```

```
            result = (tmp + (den1 >> 1)) / den1;
            mu = (result > 255) ? 8'd255 : result[7:0];
        end
        // Right slope: B <= x < C
        else if ((x >= B) && (x < C)) begin
            num2 = C - x;
            den2 = C - B;
            tmp = num2 * SCALE;
            result = (tmp + (den2 >> 1)) / den2;
            mu = (result > 255) ? 8'd255 : result[7:0];
        end
        // Peak: x == B (technically covered above, but
           explicit)
        else if (x == B) begin
            mu = 8'd255;
        end
    end

endmodule
```

Other common membership functions, such as the linear S-shaped and Z-shaped functions, are similarly simple to implement in hardware as they are also based on linear equations.

3.2.2. Inference

The inference engine serves as the decision-making core of the controller. It evaluates the fuzzy inputs generated in the fuzzification stage against the linguistic rules defined in the system's rule base. For this implementation, the logical 'AND' operator connecting the antecedents of a rule is realized using the minimum function, a standard method in fuzzy logic control.

Mathematically, for a rule with multiple conditions, such as:

$$\text{IF } x \text{ is } A \text{ AND } y \text{ is } B \text{ THEN } z \text{ is } C \tag{3.3}$$

The firing strength w of the rule is calculated as:

$$w = \min(\mu_A(x), \mu_B(y)) \tag{3.4}$$

In the SystemVerilog implementation, this logic is synthesized using combinational circuits. A dedicated function, `min3`, is created to compute the minimum of three values, facilitating rules that depend on dV, dI, and the V_{pv} operating range simultaneously. The code below illustrates the parallel evaluation of the safety rules and the fine-tuning rules. The safety rules (w_1, w_2) are triggered solely by the PV voltage level, while the fine-tuning rules (w_3 to w_8) are active only when the voltage is within the optimal range, ensuring precise tracking around the MPP. This implementation strictly adheres to the logic defined in Tables 2.2 and 2.3.

Listing 3.3. Inference SystemVerilog Implementation

```
module inference (
    // dV and dI inputs (3 each)
    input  logic [7:0] dV_mu_neg, dV_mu_ze, dV_mu_pos,
    input  logic [7:0] dI_mu_neg, dI_mu_ze, dI_mu_pos,
    // V_pv inputs (3)
    input  logic [7:0] Vpv_mu_low, Vpv_mu_opt, Vpv_mu_high,

    // Rule Firing Strengths (w1 through w8)
    output logic [7:0] w [1:8]
);

    // Internal function for 3-input MIN
    function automatic logic [7:0] min3(logic [7:0] a, logic
```

```
   [7:0] b, logic [7:0] c);
    logic [7:0] m2;
    m2 = (a < b) ? a : b;
    return (m2 < c) ? m2 : c;
endfunction

always_comb begin
    // --- GROUP A: SAFETY RULES ---
    // Rule 1: V=Low -> Big_Decrease (w1)
    w[1] = Vpv_mu_low;

    // Rule 2: V=High -> Big_Increase (w2)
    w[2] = Vpv_mu_high;

    // --- GROUP B: FINE TUNING (Only active in Optimal
       Range) ---
    // Rule 3: Pos/Pos/Opt -> Decrease
    w[3] = min3(dV_mu_pos, dI_mu_pos, Vpv_mu_opt);

    // Rule 4: Pos/Neg/Opt -> Increase
    w[4] = min3(dV_mu_pos, dI_mu_neg, Vpv_mu_opt);

    // Rule 5: Neg/Pos/Opt -> Increase
    w[5] = min3(dV_mu_neg, dI_mu_pos, Vpv_mu_opt);

    // Rule 6: Neg/Neg/Opt -> Decrease
    w[6] = min3(dV_mu_neg, dI_mu_neg, Vpv_mu_opt);

    // Rule 7: Stuck (Ze/Ze/Opt) -> HOLD
    w[7] = min3(dV_mu_ze, dI_mu_ze, Vpv_mu_opt);

    // Rule 8: Stable (Ze/Any/Opt) -> HOLD (Combining
       your two stable rules)
```

```
            // We can simplify the "Stable" logic here or list
               them separately.
            w[8] = min3(dV_mu_ze, (dI_mu_neg > dI_mu_pos ?
               dI_mu_neg : dI_mu_pos), Vpv_mu_opt);
        end
    endmodule
```

As can be seen in the `always_comb` block, the logic directly maps to the rule tables. The safety rules (w[1] and w[2]) depend only on the PV voltage, while the fine-tuning rules (w[3] through w[8]) are active only in the optimal voltage range (`Vpv_mu_opt`) and correctly implement the logic described in Table 2.3.

3.2.3. Defuzzification

The final stage of the fuzzy logic controller is defuzzification, which converts the fuzzy conclusions from the inference engine into a crisp, numerical control signal. Since this system utilizes a Sugeno-type inference model, the output membership functions are singletons (constant values) rather than distributed fuzzy sets. This simplifies the defuzzification process to a weighted average calculation, which is computationally efficient for hardware implementation.

The crisp output, representing the required change in duty cycle (ΔD), is calculated using the Center of Gravity (CoG) method for singletons:

$$\Delta D = \frac{\sum_{i=1}^{N} w_i \cdot C_i}{\sum_{i=1}^{N} w_i} \tag{3.5}$$

where w_i is the firing strength of the i-th rule, C_i is the constant output value associated with that rule, and N is the total number of rules.

The SystemVerilog implementation, shown below, defines the singleton constants corresponding to the linguistic variables `Big_Decrease` (-15), `Decrease` (-1), `Hold` (0),

`Increase` (+1), and `Big_Increase` (+15). The module accumulates the weighted sum (numerator) and the total weight (denominator) to compute the final integer division.

Listing 3.4. defuzzification SystemVerilog Implementation

```
module defuzzifier (
    input  logic [7:0] w [1:8],          // 8 Rule weights
       from Inference
    output logic signed [10:0] dDuty     // Final step size
);

    // Sugeno Constants from MATLAB
    // Index matches our Rule List:
    // w1:-15, w2:15, w3:-1, w4:1, w5:1, w6:-1, w7:0, w8:0
    localparam signed [7:0] C1 = -15; // Big Decrease
    localparam signed [7:0] C2 =  15; // Big Increase
    localparam signed [7:0] C3 =  -1; // Decrease
    localparam signed [7:0] C4 =   1; // Increase
    localparam signed [7:0] C5 =   1; // Increase
    localparam signed [7:0] C6 =  -1; // Decrease
    localparam signed [7:0] C7 =   0; // Hold
    localparam signed [7:0] C8 =   0; // Hold

    // Intermediate accumulators
    logic signed [20:0] sum_wi_ci;
    logic [15:0] sum_wi;

    always_comb begin
        // Weighted Sum: Σ(wi * Ci)
        sum_wi_ci = ( $signed({1'b0, w[1]}) * C1 ) +
                    ( $signed({1'b0, w[2]}) * C2 ) +
                    ( $signed({1'b0, w[3]}) * C3 ) +
                    ( $signed({1'b0, w[4]}) * C4 ) +
                    ( $signed({1'b0, w[5]}) * C5 ) +
```

```
                    ( $signed({1'b0, w[6]}) * C6 ) +
                    ( $signed({1'b0, w[7]}) * C7 ) +
                    ( $signed({1'b0, w[8]}) * C8 );

        // Total Weight: Σ(wi)
        sum_wi = w[1] + w[2] + w[3] + w[4] + w[5] + w[6] + w
           [7] + w[8];

        // Weighted Average (The "Defuzz")
        if (sum_wi != 0) begin
            dDuty = sum_wi_ci / $signed({1'b0, sum_wi});
        end else begin
            dDuty = 0;
        end
    end
endmodule
```

3.2.4. Duty Integration and Bumpless Transfer

The final stage of the digital control loop is the Duty Integrator. While the fuzzy inference engine determines the required *change* in the operating point (ΔD), the Pulse Width Modulation (PWM) generator requires an absolute duty cycle value to drive the power MOSFETs. This module performs the discrete-time integration of the fuzzy output, mathematically represented as:

$$D[n] = D[n-1] + \Delta D[n] \tag{3.6}$$

where $D[n]$ is the current duty cycle and $\Delta D[n]$ is the adjustment calculated by the defuzzifier.

This integration strategy serves two critical purposes:

(i) **Bumpless Transfer:** We take the duty cycle output from the general duty cycle register ($D[n-1]$) as the baseline. By adding the calculated change (ΔD) to this existing global output, the system ensures a smooth transition. This prevents discontinuities that would otherwise occur when switching between different control dynamics (e.g., Safety vs. Fine-Tuning).

(ii) **Robustness and Damping:** The integrator and damping provide the opportunity to lower oscillations and make the system robust. In the MATLAB model, this was achieved by dividing the output by a large gain factor. In the SystemVerilog implementation, where integer arithmetic prevents such division (as it would result in zero), this damping is achieved by accumulating small integer steps ($\pm 1, \pm 15$) into the high-resolution duty cycle register. This effectively lowers the impact of individual control actions, filtering out transient noise and maintaining stability around the MPP.

The SystemVerilog implementation, provided below, includes saturation logic to constrain the duty cycle within safe hardware limits. It takes the signed 8-bit output from the defuzzifier and updates the 10-bit duty cycle register on every clock cycle.

Listing 3.5. Duty Integrator SystemVerilog Implementation

```
module duty_integrator (
    input  logic        clk,
    input  logic        reset_n,
    input  logic        mppt_enable,
    input  logic signed [10:0] duty_adjust,
    input  logic [9:0]  manual_duty_in,
    output logic [9:0]  dac_value
);

    // --- CONFIGURATION ---
    // CHANGE THIS: 9 for Turbo Sim, 14 for Real Hardware
    localparam int FRAC_BITS = 6;
```

```
// Safety Headroom: 10 (Data) + FRAC_BITS + 2 (Sign/
   Overflow)
// We keep 26 bits because it is safe for both 9 and 14.
logic signed [25:0] acc;

// --- SCALED LIMITS (Calculated automatically) ---
localparam signed [25:0] MIN_ACC = 26'd307 << FRAC_BITS;
localparam signed [25:0] MAX_ACC = 26'd716 << FRAC_BITS;
localparam signed [25:0] START_VAL = 26'd512 << FRAC_BITS
   ;

always_ff @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        acc <= START_VAL;
    end else begin

        if (mppt_enable == 1'b0) begin
            // === MANUAL MODE ===
            // FIX: Use shift operator '<<' instead of
               hardcoded zeros.
            // This automatically shifts by 9 or 14
               depending on the param.
            // We cast to 26-bit signed to ensure the
               width matches 'acc'.
            acc <= signed'({1'b0, manual_duty_in}) <<
               FRAC_BITS;
        end
        else begin
            // === MPPT MODE ===
            logic signed [26:0] next_acc;
            next_acc = acc + duty_adjust;

            if (next_acc > MAX_ACC)
```

```
                    acc <= MAX_ACC;
                else if (next_acc < MIN_ACC)
                    acc <= MIN_ACC;
                else
                    acc <= next_acc[25:0];
            end
        end
    end

    // === OUTPUT ===
    // FIX: Dynamic Slicing
    // We want the 10 bits STARTING after the fractional part
       .
    // Example (FRAC=9): We want bits [18:9]
    // Example (FRAC=14): We want bits [23:14]
    assign dac_value = acc[FRAC_BITS + 9 : FRAC_BITS];

endmodule
```

It is important to note that the SystemVerilog code snippets provided in this section primarily serve to demonstrate the functional behavior and the algorithmic logic of the controller within the Cadence simulation environment. Direct implementation of arithmetic operations such as division—used here for the center-of-gravity defuzzification—can consume excessive logic resources and timing budget during synthesis. In a practical hardware implementation, these operations are often optimized or approximated using shifts, look-up tables, or specific architectural constraints to minimize area and power. Further details on the optimized digital implementation and synthesis considerations are discussed in Chapter 5.

3.3. Simulation Results

The functional verification of the SystemVerilog implementation was performed within the Cadence Virtuoso environment. The simulation was run for 180 ms, with the MPPT enabled at 0.3 ms.

As discussed in the duty integrator section, the selection of the damping fraction is critical. A fraction of 6 was selected; a higher value (e.g., 15) would result in excessive settling time for transient ringing. To further stabilize the inputs, V_{pv}, I_{pv}, and I_{out} are processed through a moving average block with a window width of 200 μs.

Figure 3.3 illustrates the system's response. The extracted input power (P_{in}) settles at 227 W, which is close to the theoretical maximum of 231.4 W listed in Table 3.1. The average PV voltage (V_{pv_avg}) stabilizes at 30.48 V, compared to the theoretical MPP voltage of 30.29 V. The output power (P_{out}) is measured at 217 W. This results in a PV efficiency (P_{in}/P_{vmax}) of $227/231.4 \approx 98.1\%$ and a converter efficiency (P_{out}/P_{in}) of $217/227 \approx 95.6\%$. Despite these minor deviations, the model demonstrates robust operation and successful tracking.

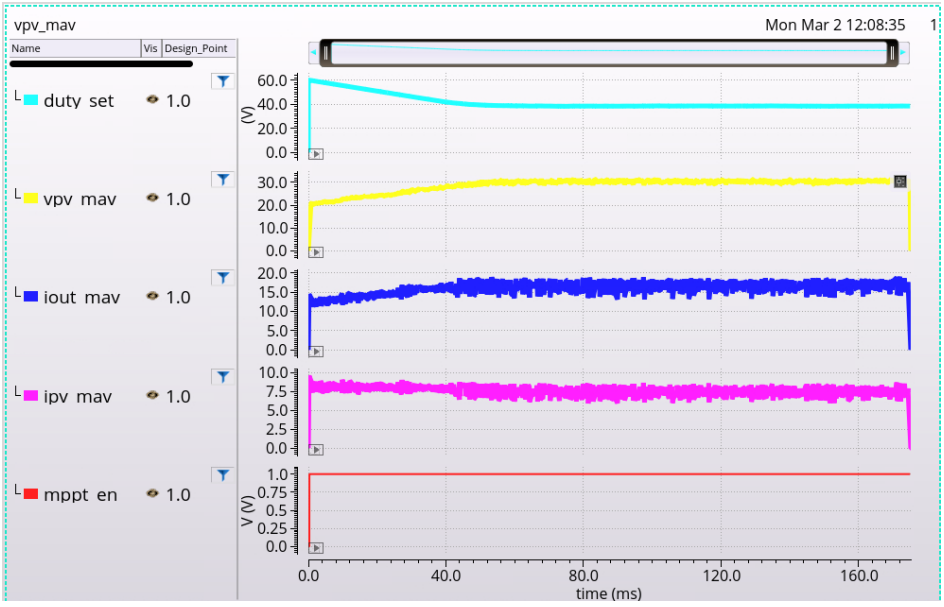


Figure 3.3. Cadence Mixed-Signal Simulation Results showing MPP Tracking.

3.4. System Block Diagram

The complete system architecture integrates the digital control logic with the analog power and sensing components. Figure 3.4 illustrates the detailed block diagram, showing the signal flow from the PV panel sensors through the mixed-signal interface to the Fuzzy Logic Controller, and the resulting control signal driving the Buck Converter.

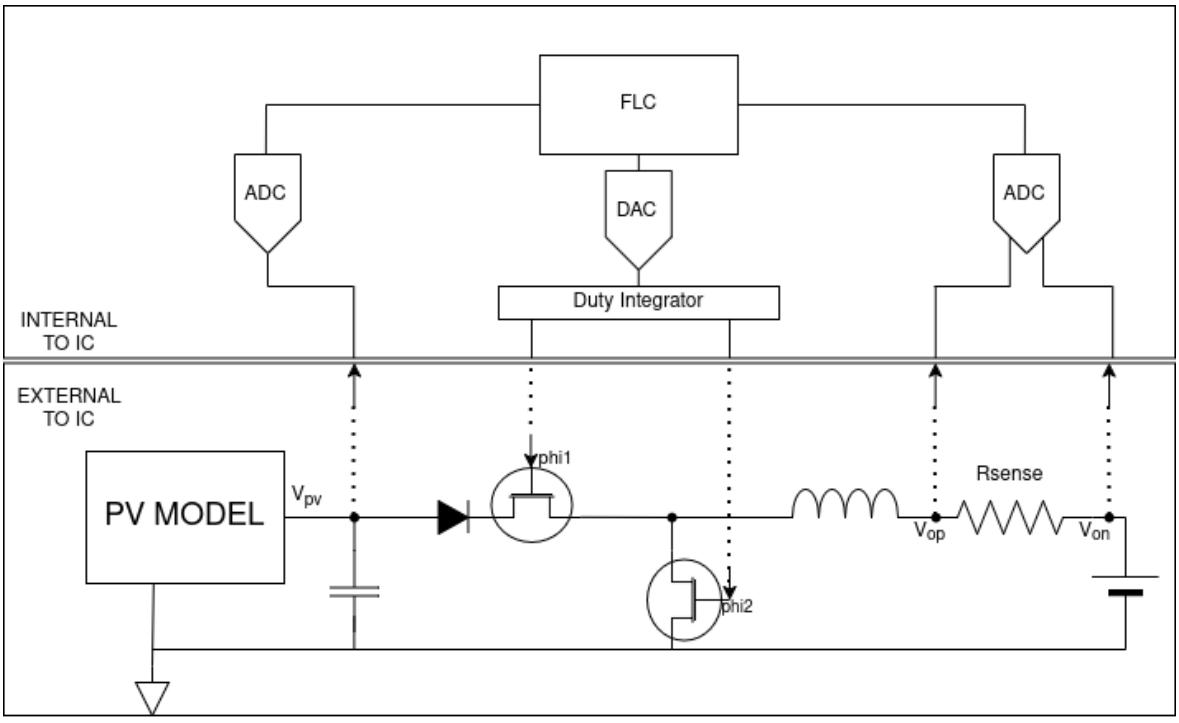


Figure 3.4. System Block Diagram.

To ensure precise tracking and stable operation, the system is decomposed into the following hierarchical requirements:

- **Digital Controller**
  - **Fuzzy Logic Controller:** Implements the Sugeno inference engine, rule base, and duty integrator.
- **Analog / Mixed-Signal Interface**
  - **Analog-to-Digital Converters (ADC)**
    - *Resolution:* 10-bit

    * *Sampling Rate:* 20 kSPS
    * *Function:* Digitizes V_{pv} and I_{pv} for the controller.
  - **Digital-to-Analog Converter (DAC)**
    * *Resolution:* 10-bit
    * *Function:* Converts the digital output of the FLC to an analog signal to drive the duty controller.
- **Analog Support & References** (Required for ADC/DAC precision)
  - **Low Dropout Regulator (LDO):** Provides a clean, regulated supply voltage to isolate the sensitive analog blocks from switching noise.
  - **Bandgap Reference (BGR):** Generates a stable, temperature-independent voltage reference required by the LDO and ADCs.
- **Power Stage & Drivers**
  - **Duty Generator (PWM Adjuster):** This block compares the analog control voltage from the DAC against a high-frequency triangular waveform to generate the PWM signal. The resulting duty cycle is proportional to the control voltage, directly modulating the power flow through the buck converter.
  - **High-Side Switch Driver:** A specialized circuit required to drive the gate of the high-side N-MOSFET. Since the MOSFET's source is not ground-referenced but connected to the switching node, this driver must provide a gate voltage that is higher than the input voltage (V_{in}) to turn the switch on. This is typically achieved using a bootstrap circuit.
  - **Low-Side Switch Driver:** This circuit drives the gate of the low-side synchronous N-MOSFET. As its source is connected to ground, the driver provides a simpler 0V to V_{DD} logic-level signal, but it must be capable of sourcing and sinking high peak currents to charge and discharge the gate capacitance quickly, minimizing switching losses.

4. ANALOG IMPLEMENTATION

4.1. ADC

The analog-to-digital converter (ADC) is a critical component in the mixed-signal feedback loop, responsible for digitizing the PV voltage and current for the digital controller. For this system, a fully differential 12-bit Successive Approximation Register (SAR) ADC was chosen due to its excellent power efficiency and suitability for medium-speed, high-resolution applications. The fully differential architecture offers superior noise immunity, rejecting common-mode noise which is prevalent in switching power converter environments.

The ADC operates asynchronously, meaning the internal bit-cycling is triggered by the completion of the previous bit decision rather than a high-speed external clock. This simplifies the clocking requirements of the system.

Figure 4.1 illustrates the top-level architecture of the implemented SAR ADC.

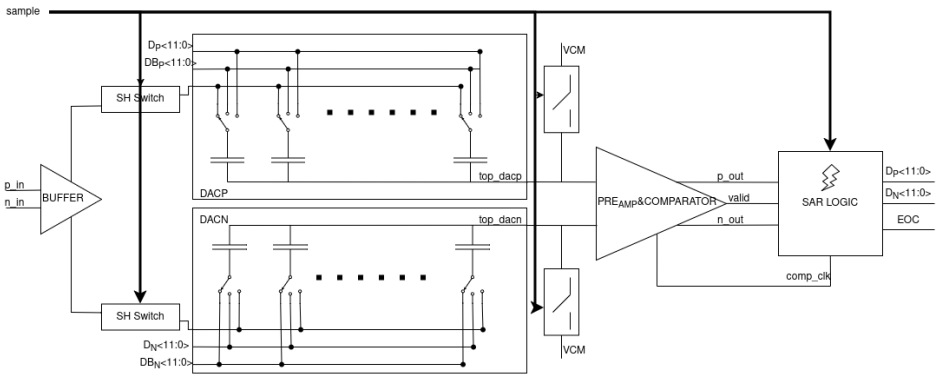


Figure 4.1. Fully Differential 12-bit Asynchronous SAR ADC Architecture.

To achieve the required linearity and speed, the ADC is composed of several critical sub-blocks, which are analyzed in detail in the following sections:

4.1.1. Sample and Hold Switch

The Sample and Hold (S/H) switch is the front-end of the ADC, responsible for acquiring the analog input signal onto the CapDAC array. To achieve high linearity over a wide input common-mode range, a bootstrapped switch topology is employed. The design is based on the well-known bootstrapped switch architecture proposed by Razavi [5], which ensures a constant gate-to-source voltage (V_{gs}) for the main sampling transistor. This minimizes signal-dependent charge injection and maintains a low, constant on-resistance, which is critical for achieving 12-bit linearity.

To further enhance robustness, the bulk connections of the transistors are critical. The final implementation of the switch is shown in Figure 4.2. The design of such high-performance switches is a well-studied area in CMOS analog design [5, 6].

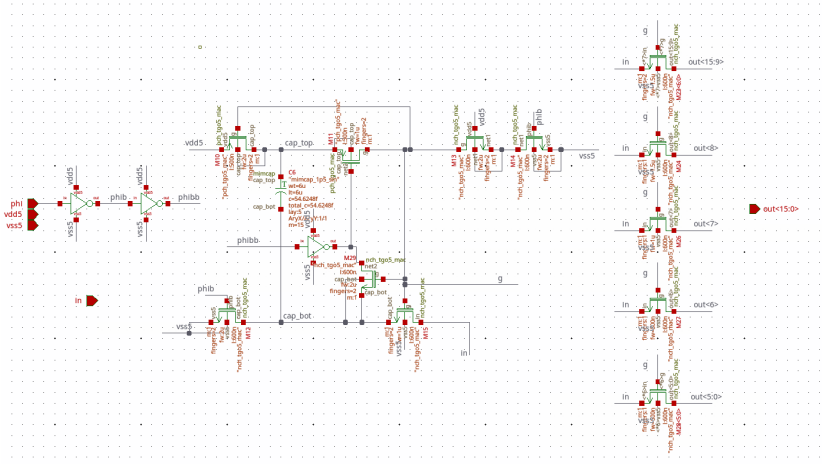


Figure 4.2. Bootstrapped Sample and Hold Switch Implementation.

The parasitic capacitance on the switching node was calculated as 60 fF in the typical process corner. Consequently, a 500 fF bootstrap capacitor was selected. Although this results in a gate-source voltage (V_{gs}) drop of approximately 10% due to charge sharing, the resulting on-resistance of the MOSFET is still sufficiently low to charge the ∼25 pF sampling capacitor within the 20 kHz sampling period.

Linearity is the main metric for this device, as any distortion introduced at the

sampling stage directly limits the overall accuracy of the ADC. Figure 4.3 shows the 4096-point Discrete Fourier Transform (DFT) of the signal passed through the switch. The spectrum demonstrates the high linearity of the design, with harmonic components significantly suppressed relative to the fundamental tone.

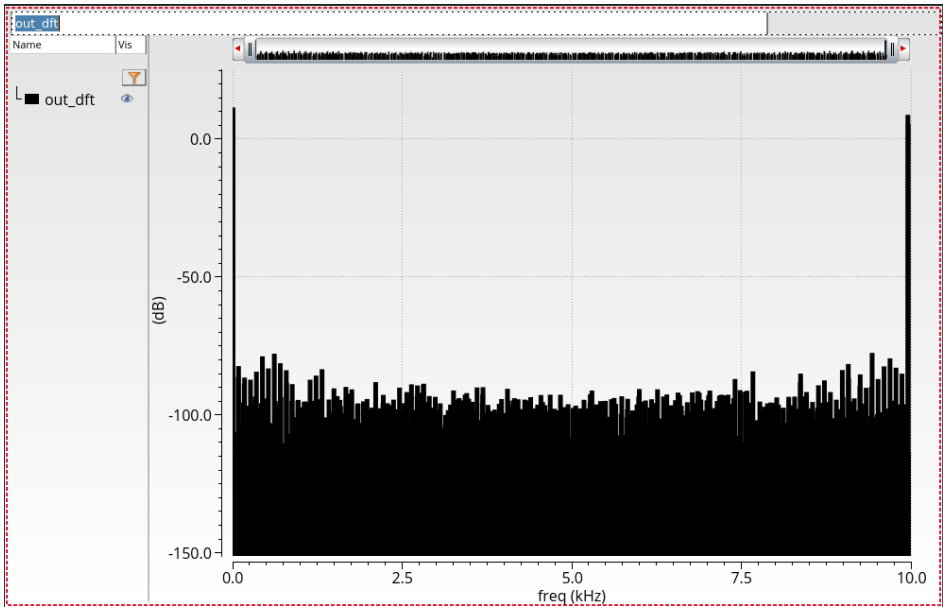


Figure 4.3. DFT of the Signal Passed Through the Sample and Hold Switch.

Table 4.1. Sample and Hold Switch Performance Metrics (Input: 0-5 V FS)

Metric	**Value**
THDB3	-89.12 dB
SFDR	82.2 dB

4.1.2. CapDAC

The Capacitor-DAC (CapDAC) utilizes a split-capacitor architecture to reduce the total capacitance and silicon area. Instead of a conventional 2^{12} unit capacitor array, the design employs a 6-bit MSB and 6-bit LSB split. Although the minimum Metal-Insulator-Metal (MIM) capacitor allowed by the PDK is 6 fF, a unit capacitor of 60 fF is used in this architecture. Following the equivalent value method described in [7], a bridge capacitor of 2C (twice the unit capacitance) was chosen, reducing the total number of unit capacitors to 128. To ensure proper binary weighting, an

additional dummy capacitor is required. The calculation for this dummy capacitor was performed using the MATLAB script in Listing 4.1, which implements the method from [7], and resulted in a value of 62C. This configuration significantly improves the settling time and bandwidth of the ADC.

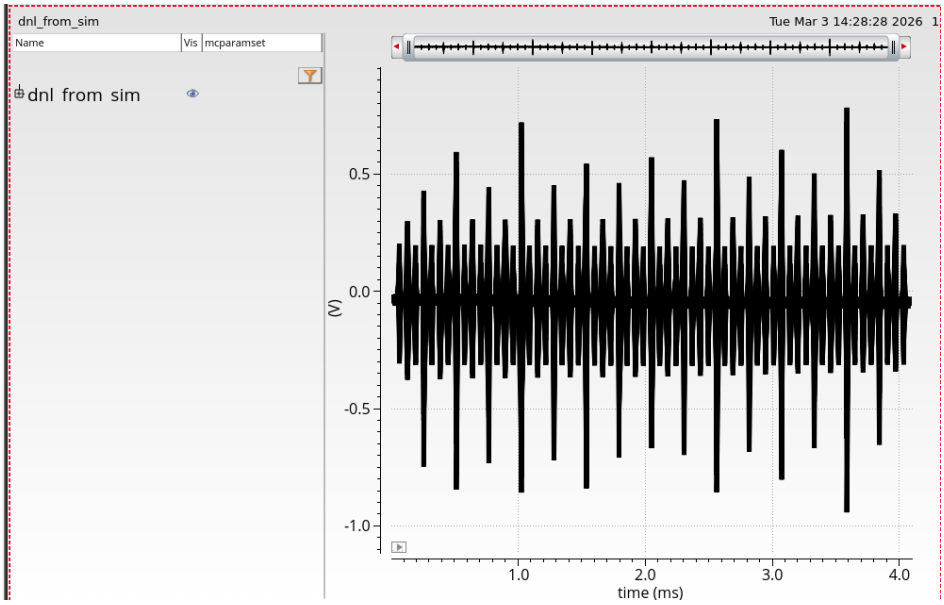
Listing 4.1. Split Capacitor Array Calculation Script

```
clear; clc;

% ==========================================
% STEP 1: DIRECT CONFIGURATION
% ==========================================
N = 12;      % Total Bits
L = 6;       % LSB Bits
M = 6;       % MSB Bits
m = 2;       % Bridge Capacitor Ratio (Cbri = m * C)
C_unit = 1; % Normalized unit cap

% ==========================================
% STEP 2: EWM CALCULATIONS (Eq. 4 logic)
% ==========================================
% Formula for Cc: (2^L - 1)*C_bri = 2^L*C + Cc
C_bri = m * C_unit;
C_lsb_array_sum = 2^L * C_unit; % e.g., 64C for 6-bit LSB
C_c = (2^L - 1) * C_bri - C_lsb_array_sum;

fprintf('--- Configuration Results ---\n');
fprintf('Resolution: %d-bit (%d LSB + %d MSB)\n', N, L, M);
fprintf('Bridge Cap (C_bri): %.1f * C\n', m);

if C_c < 0
    fprintf('WARNING: Calculated Cc is %.1f * C.\n', C_c);
    fprintf('A negative capacitor is physically impossible!\n
       ');
```

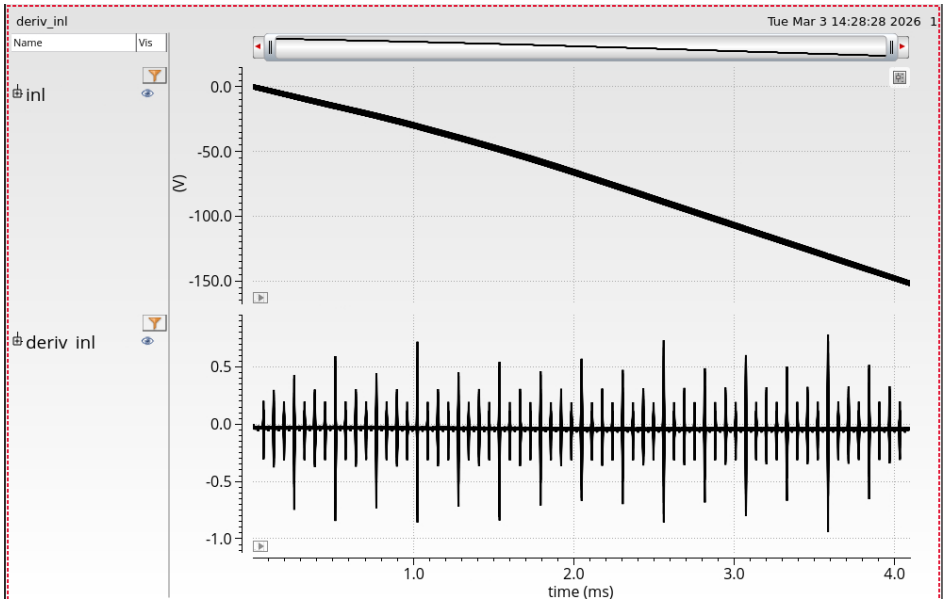
```
    fprintf('Reason: C_bri=1 is too small for a 6+6 split.
       Try m >= 1.016 or m = 2.\n\n');
else
    fprintf('Adjustment Cap (Cc): %.1f * C\n\n', C_c);
end

% ==========================================
% STEP 3: WEIGHT VERIFICATION
% ==========================================
% Generate LSB binary weights: [1, 2, 4, 8, 16, 32...]
lsb_bits = 2.^(0:L-1);
msb_bits = 2.^(0:M-1);

% Calculate Attenuation Factor (alpha)
% alpha = Cbri / (Cbri + C_lsb_total + Cc)
C_lsb_total = C_lsb_array_sum + C_c;
alpha = C_bri / (C_bri + C_lsb_total);

% Final Weights
w_lsb = alpha * lsb_bits;
w_msb = msb_bits;
all_weights = [w_lsb, w_msb];

% Linearity Check: The gap between the LSBs and MSBs
% Ideally, MSB[0] should be exactly 2x the LSB[max]
gap = w_msb(1) / w_lsb(end);
fprintf('--- Linearity Verification ---\n');
fprintf('Weight Ratio (MSB_min / LSB_max): %.4f\n', gap);
if abs(gap - 2) < 1e-6
    fprintf('RESULT: Binary Weighting is PERFECT.\n');
else
    fprintf('RESULT: Binary Weighting is BROKEN.\n');
end
```

```
% ==========================================
% STEP 4: PLOT TRANSFER FUNCTION
% ==========================================
num_steps = 1000;
vin = linspace(0, sum(all_weights), num_steps);
digital_out = zeros(size(vin));

for i = 1:num_steps
    v_rem = vin(i);
    code = 0;
    for b = N:-1:1
        if v_rem >= all_weights(b)
            v_rem = v_rem - all_weights(b);
            code = code + 2^(b-1);
        end
    end
    digital_out(i) = code;
end

figure;
plot(vin, digital_out, 'LineWidth', 1.5);
title(sprintf('%d-bit ADC (%d+%d Split, m=%.1f)', N, L, M, m)
   );
xlabel('Analog Input'); ylabel('Digital Output Code');
grid on;
```

The linearity of the CapDAC is paramount for the overall ADC performance. The two key metrics are Differential Non-Linearity (DNL), which measures the deviation in step width from the ideal 1 LSB, and Integral Non-Linearity (INL), which is the cumulative sum of DNL errors. Figure 4.4 shows the simulated DNL and INL of the designed CapDAC.



(a) DNL



(b) INL and its derivative

Figure 4.4. CapDAC Linearity Simulation Results.

The results shown in Figure 4.4 are obtained from a Monte Carlo simulation of 200 runs. The Differential Non-Linearity (DNL) is observed to be up to ±1 LSB, which is a natural characteristic in split-capacitor arrays due to bridge capacitor sensitivity. However, since the target Effective Number of Bits (ENOB) for this application is 10 bits, this performance is considered acceptable. The Integral Non-Linearity (INL) is

also within acceptable limits, exhibiting a nearly constant gain error. This is further illustrated by the derivative of the INL, plotted as $d(INL)/dt \times 1\mu s$ (corresponding to the LSB step time). Finally, to further improve the ADC performance after fabrication, the bridge capacitor is implemented with trim capacitors consisting of 6 fF unit cells. Additionally, a calibration capacitance of $0.1 \times C_{unit}$ is added.

4.1.3. Preamplifier

In a high-resolution SAR ADC, the dynamic comparator can inject significant kickback noise back onto the sensitive CapDAC nodes when it fires. This charge injection can corrupt the voltage stored on the capacitor array, leading to erroneous bit decisions. To mitigate this effect and improve the overall linearity, a preamplifier is placed between the CapDAC and the comparator.

The preamplifier serves two primary functions:

(i) **Isolation:** It acts as a buffer, shielding the delicate analog nodes of the CapDAC from the comparator's kickback noise.

(ii) **Gain:** It provides a moderate amount of voltage gain, amplifying the small differential voltage from the CapDAC. This relaxes the sensitivity requirements for the subsequent comparator, reducing its effective input-referred offset and noise.

The implemented preamplifier, shown in Figure 4.5, is a fully differential amplifier that utilizes top-plate sampling. A fully differential topology is used to maintain high common-mode noise rejection throughout the signal path. To stabilize the output common-mode voltage, which is otherwise undefined in a differential amplifier, a dedicated Common-Mode Feedback (CMFB) circuit is employed. The reference voltage for the CMFB is set to VDD/2, which is generated by a simple resistive divider filtered by a capacitor to reduce noise; this VDD/2 reference will also be used by the top-plate biasing LDO. The CMFB circuit senses the average output voltage, compares it to this

reference, and adjusts the gate voltage of the PMOS load transistors to maintain a stable DC operating point.

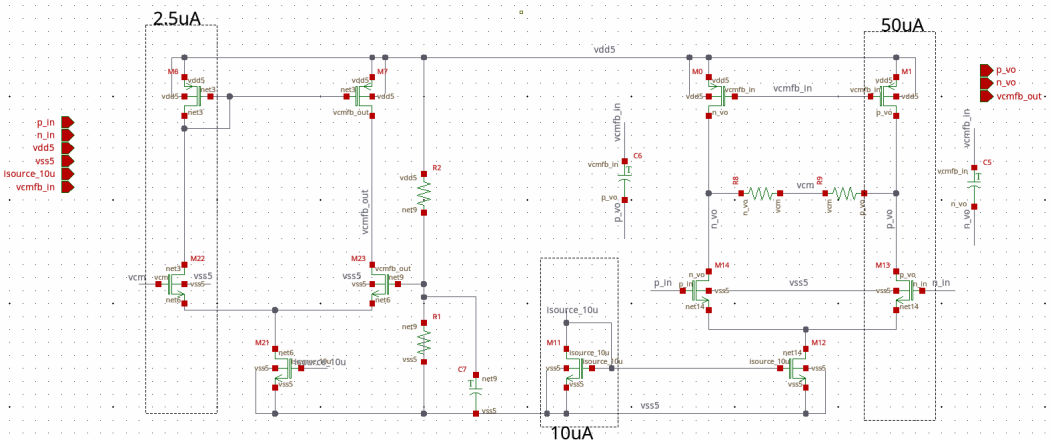


Figure 4.5. Fully Differential Preamplifier with CMFB Circuit.

4.1.3.1. Simulation Results. The performance of the preamplifier was verified through extensive simulations. The key metrics evaluated include differential and common-mode stability, input-referred offset, and noise performance.

CMFB Stability Analysis. Ensuring stability is crucial for the preamplifier. The primary concern is the stability of the Common-Mode Feedback (CMFB) loop, which must be analyzed to prevent oscillations and ensure a stable output common-mode level.

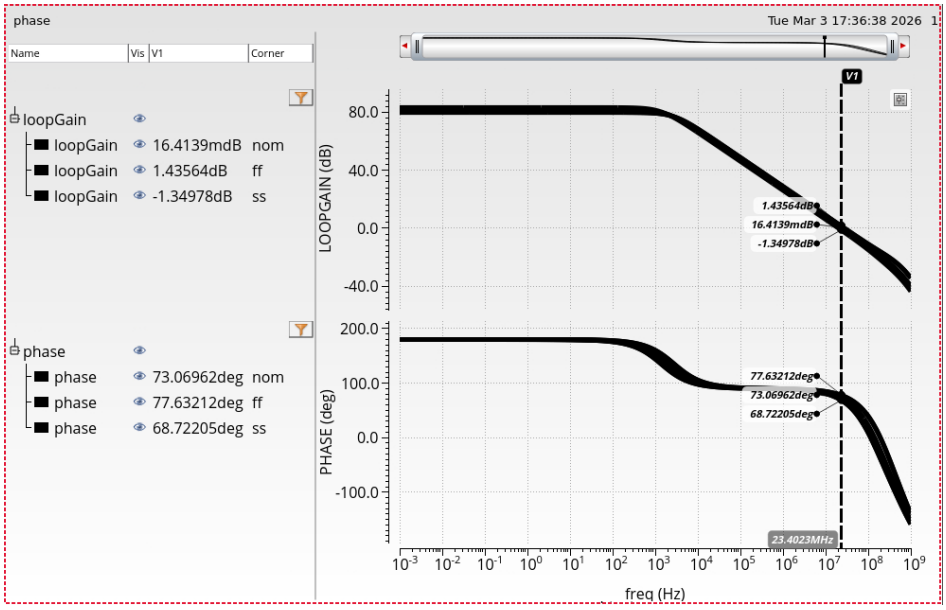


Figure 4.6. CMFB Loop Stability Simulation.

Common Mode Voltage. The output common-mode voltage is maintained by the CMFB loop. Figure 4.7 shows the Monte Carlo simulation results for the output common-mode voltage, demonstrating the effectiveness of the CMFB circuit across variations.

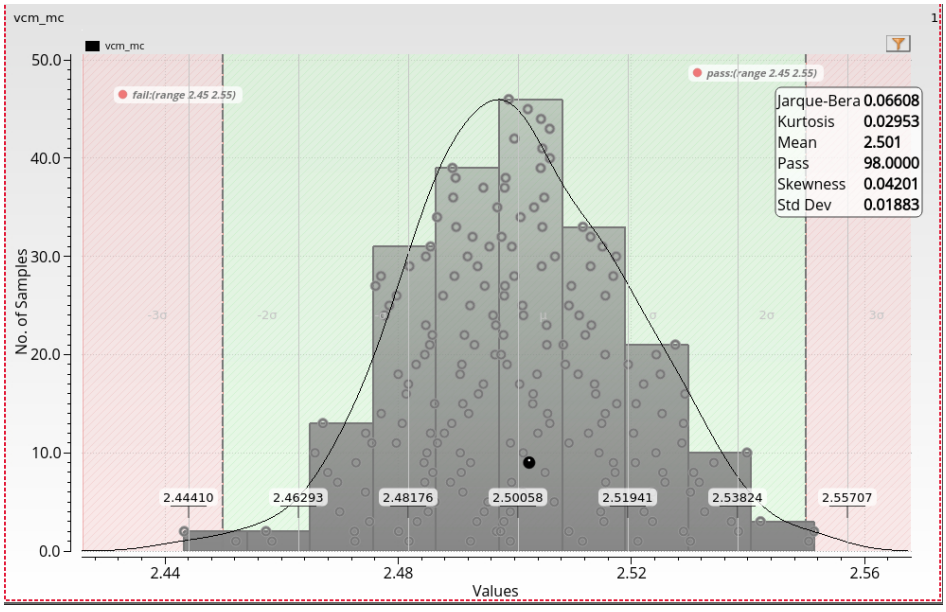


Figure 4.7. Output Common Mode Voltage Monte Carlo Simulation.

DIFF OPEN LOOP Analysis. The open-loop differential path is also analyzed, mainly to observe its phase delay and ensure it settles quickly without excessive ringing.

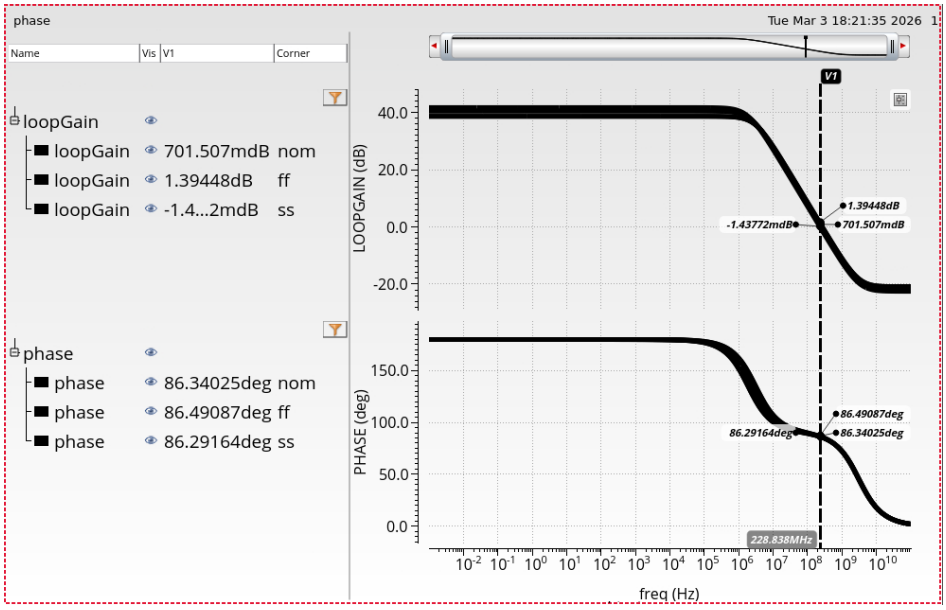


Figure 4.8. Differential Open Loop AC Response.

Offset. The input-referred offset directly impacts the effective resolution of the ADC.

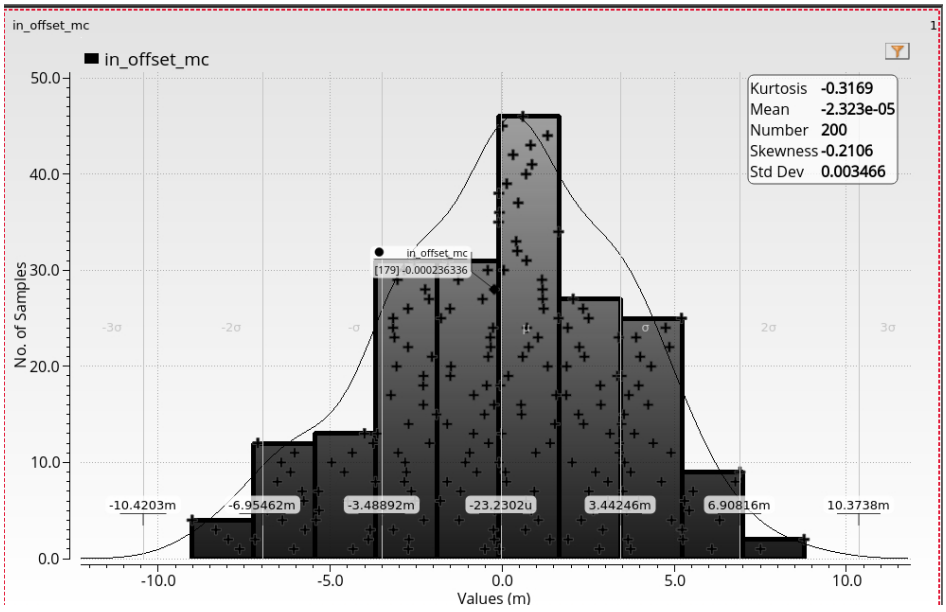


Figure 4.9. Input Referred Offset Monte Carlo Simulation.

Noise. The input-referred noise is another critical factor limiting the ADC's lsb resolution. The noise is integrated up to 37 MHz and calculated as 76.97 μV_{rms}.

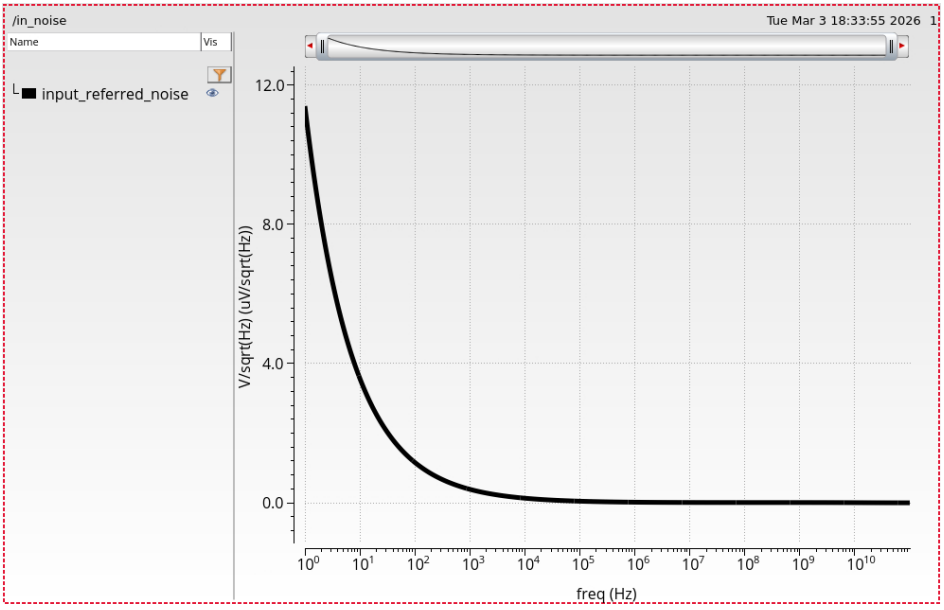


Figure 4.10. Input Referred Noise Simulation.

Table 4.2. Preamplifier Performance Summary

Parameter	**Value**
Differential Gain	38.7-41.8 dB
Differential Bandwidth (BW)	1.85-3.1 MHz
Differential GBW	228.8-269 MHz
CMFB Phase Margin	71.7-75.43°
CMFB GBW	20.1-27.6 MHz
Input Referred Offset (σ)	-+10 mV
Input Referred Noise (RMS)	76.97 μV
Output Common Mode Voltage (V_{ocm})	2.45-2.55 V

4.1.4. Comparator

The comparator is the decision-making block of the SAR ADC. To achieve high speed with zero static power consumption, a dynamic latch topology is employed. The design utilizes the StrongARM latch architecture, a widely adopted topology in

low-power ADCs. The implementation incorporates the insights and modifications presented by Razavi [8] to optimize the regeneration speed and input-referred offset.

4.1.4.1. Offset and Noise. Measuring the input-referred offset is straightforward using Monte Carlo simulations to find the trip point distribution. However, for noise performance, the dynamic nature of the latch requires a specific simulation strategy. The input-referred noise was analyzed using the transient noise simulation methodology described in [9].

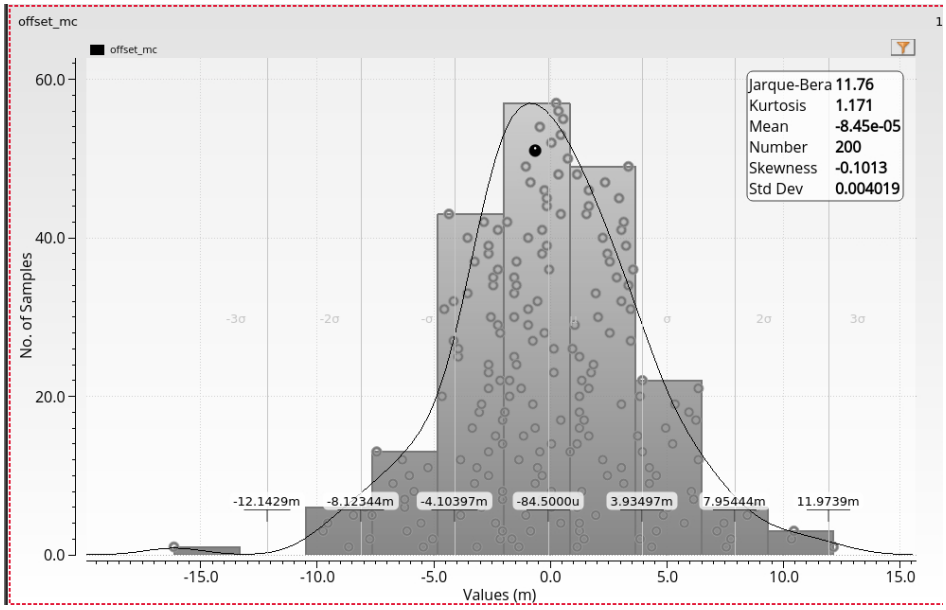


Figure 4.11. Comparator Input Referred Offset Monte Carlo Simulation.

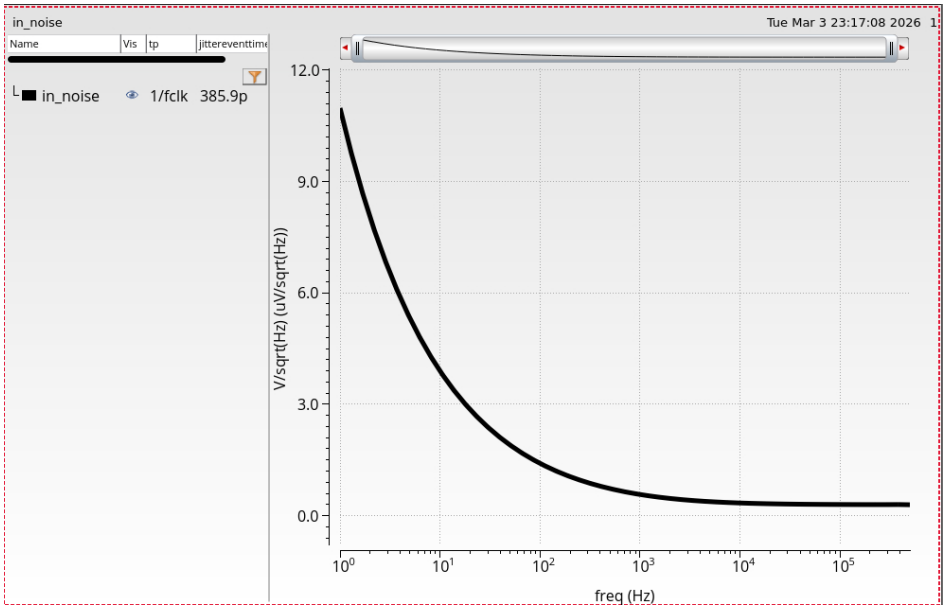


Figure 4.12. Comparator Input Referred Noise Simulation.

Table 4.3. Comparator Performance Summary

Parameter	**Value**
Input Referred Offset (σ)	-15-12 mV
Input Referred Integrated Noise (RMS)	216 μV

It is important to note that these values are referred to the input of the comparator. Since they are divided by the gain of the preamplifier when referred to the ADC input, their impact on the overall system performance is significantly reduced and considered negligible.

4.1.5. SAR Logic

The Successive Approximation Register (SAR) logic is the digital core that orchestrates the binary search algorithm. To maximize operation speed and avoid the need for a high-frequency oversampling clock, the logic is implemented using an asynchronous topology. The internal state transitions are self-timed, triggered by the ready signal from the comparator rather than a global clock edge.

The system operates across two voltage domains. The analog front-end, including the switches and capacitor array, operates at the 5V I/O voltage to maximize the signal-to-noise ratio. However, the complex digital logic is implemented using 1V core transistors to reduce dynamic power consumption and silicon area. Consequently, the comparator output is level-shifted down to 1V before being processed by the SAR engine. The resulting control signals are then level-shifted back up to 5V to drive the CapDAC switches.

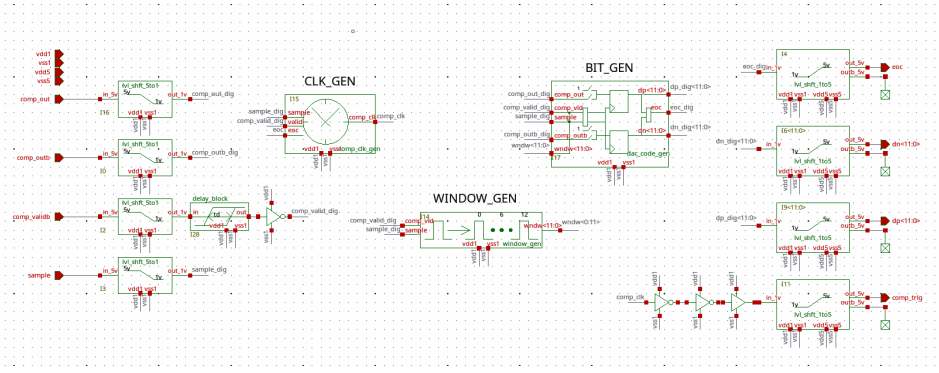


Figure 4.13. Asynchronous SAR Logic Overview with Level Shifters.

4.1.5.1. Window Generator. The window generator is responsible for orchestrating the timing of the bit trials. It generates 12 distinct window outputs, corresponding to the 12 bits of resolution. These signals are mutually exclusive; only one window bit is high at any given instance, indicating the specific bit currently being processed. This ensures the sequential operation of the SAR algorithm.

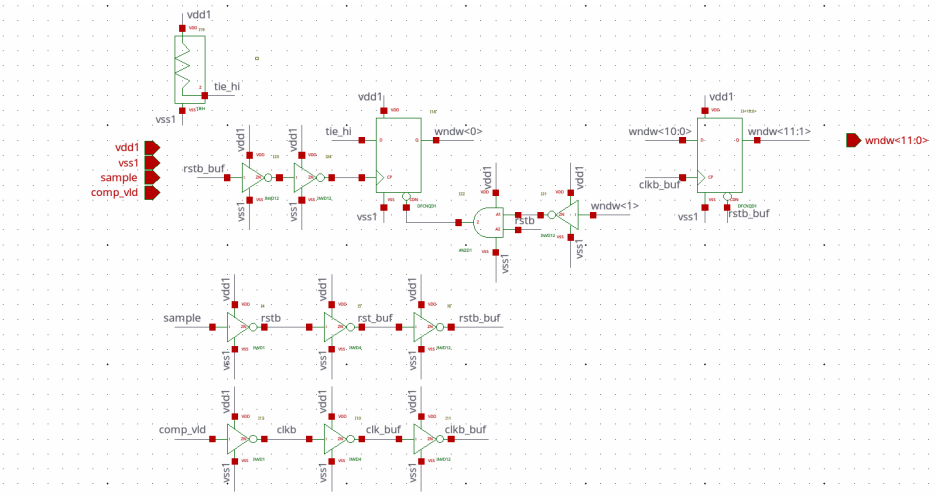


Figure 4.14. Window Generator Logic Implementation.

4.1.5.2. Comparator Clock Generator. The comparator clock is generated asynchronously to maximize speed and eliminate the need for a high-frequency external clock. The process initiates with the sampling signal, which triggers the first clock pulse. Subsequent clock pulses are self-timed; they are generated upon the receipt of a 'valid' signal from the comparator, indicating that a decision has been made. This loop continues, generating a new clock edge for each bit trial, until the End of Conversion (EOC) signal is asserted, which inhibits further clock generation. Ideally, the EOC signal should directly reset the flip-flops to minimize power consumption, but this optimization was not implemented.

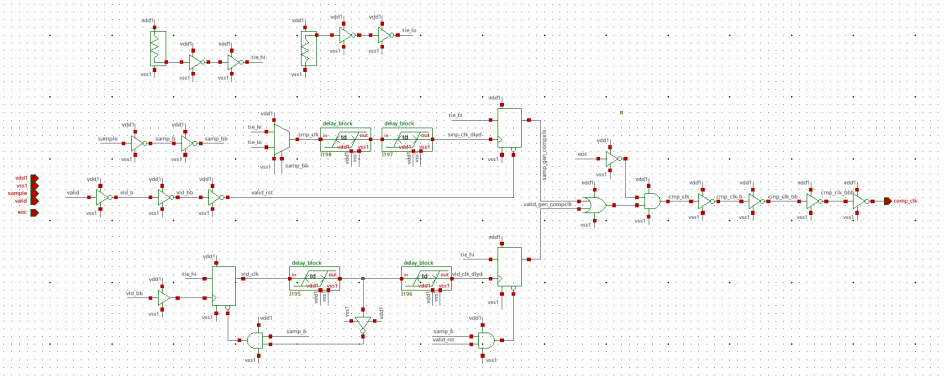


Figure 4.15. Asynchronous Comparator Clock Generation Logic.

A potential enhancement for this architecture is the implementation of adaptive

clock timing. Generally, the MSB capacitors in the DAC require a longer settling time compared to the LSB capacitors due to the larger voltage steps and stricter accuracy requirements. Conversely, the LSBs settle much faster. By leveraging the window bits, which track the current conversion step, the system can dynamically select different delay paths within the clock generation loop. This approach allocates more time for MSB decisions and speeds up LSB decisions, thereby optimizing the total conversion time. The bottom half of the circuit can be easily modified to implement this, but it is not necessary. To better illustrate the timing of the asynchronous operation, a simplified timing diagram is provided in Figure 4.16.

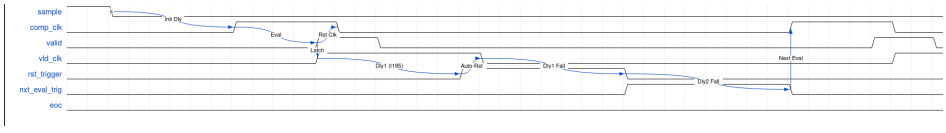


Figure 4.16. Timing Diagram of the Asynchronous Operation.

4.1.5.3. Bit Generator. The bit generator controls the sequential switching of the capacitor array, moving from the MSB to the LSB. There are no special design intricacies for this block; a direct set-and-down logic is implemented.

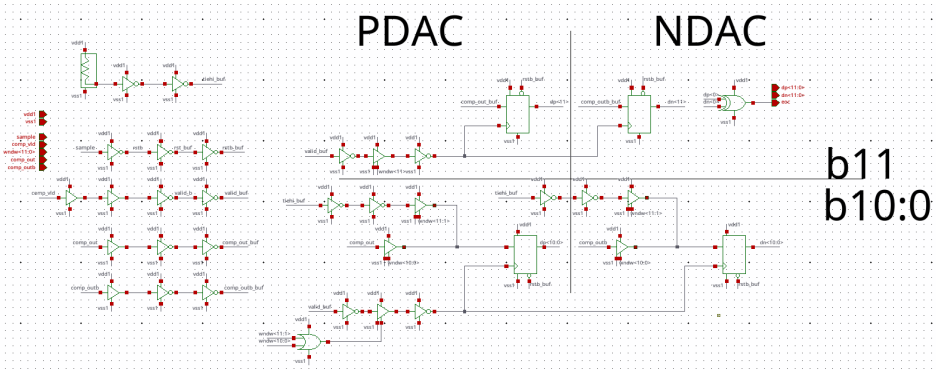


Figure 4.17. Bit Generator Logic Implementation.

4.1.6. ADC Top-Level Performance

The overall performance of the 12-bit SAR ADC was evaluated, considering the impact of noise, linearity, and process variations.

Given the 5V input range, the LSB size is approximately 1.22 mV. The combined input-referred noise from the preamplifier and comparator is significantly lower than the quantization noise level. Consequently, the system is not noise-limited, and the 12-bit resolution is not compromised by thermal noise.

The primary source of error is the Differential Non-Linearity (DNL) arising from the split-capacitor DAC array. As previously discussed, the DNL can approach ± 1 LSB. However, since the target Effective Number of Bits (ENOB) for this application is 10 bits, this linearity performance is considered acceptable. Nevertheless, as a precautionary measure to further minimize DNL errors, calibration capacitances were incorporated around the bridge capacitor.

Regarding the offset voltage introduced by the preamplifier and comparator, its impact on the MPPT algorithm is minimal. The system operates in a relatively slow-changing environment (solar irradiance and temperature), and the control logic relies primarily on the differential changes in voltage (dV) and current (dI) rather than absolute precision. Since the offset appears as a constant DC error across consecutive samples, it cancels out during the differential calculation, ensuring that the tracking direction remains accurate.

Figure 4.18 presents the 4096-point FFT of the ADC output, illustrating the dynamic performance. The analysis was performed using a Blackman-Harris window with a sampling frequency of 20 kSPS. The input frequency was chosen as $f_{in} = \frac{2039}{4096} f_s$, placing it close to the Nyquist frequency. The input signal was a differential sinusoid with a differential peak-to-peak swing of ± 8 V. For the dynamic performance calculations, the peak saturation level was set to 5 V, and the first 10 harmonics were considered.

For the Slow-Slow (ss) and Fast-Fast (ff) process corners, a 256-point FFT was utilized. The primary purpose of this analysis was to verify the functional robustness of the asynchronous logic and to confirm that process variations do not cause timing

issues, rather than to obtain high-precision dynamic performance metrics. Table 4.4 summarizes the key performance metrics across different process corners.

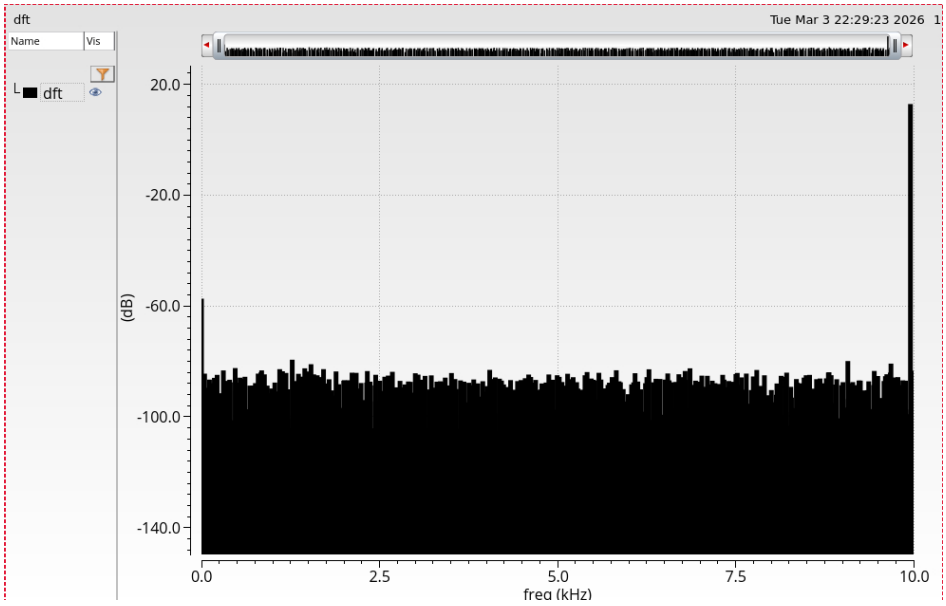


Figure 4.18. 4096-Point FFT of the ADC Output.

Table 4.4. ADC Performance Across Process Corners

Corner	**SFDR (dB)**	**SNR (dB)**	**SINAD (dB)**	**ENOB (bits)**
Typical (tt)	98.12	72.03	72	11.99
Slow-Slow (ss)	88.18	68.91	70.53	11.96
Fast-Fast (ff)	87.6	69.36	70.89	12.02

4.2. DAC

The digital-to-analog converter used in the system.

4.3. LDO

The Low-Dropout regulator design and implementation.

4.4. Duty Cycle Generator

The circuit for generating the duty cycle for the buck converter.

5. DIGITAL IMPLEMENTATION

5.1. Fuzzy Logic

A detailed description of the digital implementation of the fuzzy logic controller.

5.2. SPI Interface

The implementation of the Serial Peripheral Interface (SPI) for communication will be detailed here.

6. CONCLUSION

The conclusions of the thesis should come here.

REFERENCES

1. Erickson, R. and D. Maksimovic, *Fundamentals of Power Electronics*, Springer, 2nd edn., 2001.

2. Rashid, M. H., *Power Electronics Handbook*, Butterworth-Heinemann, 4th edn., 2017.

3. Instruments, T., *Basic Calculation of a Buck Converter*, 2015, application Report SLVA477B.

4. Soto, W. D., S. Klein and W. Beckman, "Improvement and validation of a model for photovoltaic array performance", *Solar Energy*, Vol. 80, No. 1, pp. 78–88, 2006.

5. Razavi, B., "The Bootstrapped Switch", *IEEE Solid-State Circuits Magazine*, Vol. 7, No. 3, pp. 12–15, 2015.

6. Abo, A. M. and P. R. Gray, "A 1.5-V, 10-bit, 14.3-MS/s CMOS pipeline analog-to-digital converter", *IEEE Journal of Solid-State Circuits*, Vol. 34, No. 5, pp. 599–606, 1999.

7. Fan, H., P. Lei, Q. Feng and Q. Wei, "Optimized Split Capacitive Array in 16-Bit SAR ADC with Redundancy", *Circuits, Systems, and Signal Processing*, Vol. 42, No. 3, pp. 1264–1278, 2023.

8. Razavi, B., "The StrongARM Latch", *IEEE Solid-State Circuits Magazine*, Vol. 5, No. 2, pp. 12–17, 2013.

9. Systems, C. D., "Noise Analysis of a Dynamic Comparator", Cadence Chalk Talk, 2016, https://www.cadence.com/content/dam/cadence-www/global/en_US/videos/tools/custom-_ic_analog_rf_design/NoiseAnalyisposting201612Chalk%20Talk.pdf.

10. Ang, A.-S., W. Tang and S. Ross, *Probability Concepts in Engineering*, John Wiley & Sons, New Jersey, 2007.

11. Maiers, J. and Y. Sherif, "Application of Fuzzy Set Theory", *IEEE Transactions on Systems, Man, and Cybernatics*, Vol. 15, No. 1, pp. 41–48, 1985.

12. Brown, D. and D. Clair, "Integrated R-transportation", *14th International Conference of Transportation Logistics*, Istanbul, Turkey, 2008.

13. İşgüder, E., *Human Computer Interaction in the 21st Century*, Master's Thesis, Boğaziçi University, 2015.

14. Sarp, B., "Observations", 2013, http://sarp.com.tr/obs.html, accessed on November 21, 2015.

15. Shatner, W., L. a. Christel, D. A. Borkholder and S. J. Young, "Apparatus for Analysis of a Nucleic Acid Amplification Reaction", U.S. Patent 6942971 B2, 13 September 2005, https://www.google.com/patents/US6942971, accessed on December 12, 2017.

16. "The MakerBot Replicator Desktop 3D Printer (Fifth Generation Model) User Manual", MakerBot Industries, Brooklyn, NY, 2014.

17. J., N. and C. Nettelblad, "TannyFS: Opportunistically Maximizing I/O Throughput Exploiting the Transactional Nature of Batch-Mode Data Processing", ArXiv:1612.06830 [cs], 2016.

APPENDIX A: APPENDIX A

This is the first appendix.